

Code appendix | evaluation_framework.py

Data audit, train/validation split, mean baseline, exact sMAPE

```
001  """Training-only data audit and reproducible naive baseline for individual prediction.
002
003  Usage: python evaluation_framework.py
004  Outputs are written to audit_outputs/. No prediction file is generated.
005  """
006
007  from __future__ import annotations
008
009  import json
010  from pathlib import Path
011
012  import numpy as np
013  import pandas as pd
014  from sklearn.model_selection import train_test_split
015
016
017  ROOT = Path(__file__).resolve().parent
018  OUTPUT = ROOT / "audit_outputs"
019  TARGETS = (
020      "Q0_TOTAL_ASSETS",
021      "Q0_TOTAL_LIABILITIES",
022      "Q0_TOTAL_STOCKHOLDERS_EQUITY",
023      "Q0_GROSS_PROFIT",
024      "Q0_COST_OF_REVENUES",
025      "Q0_REVENUES",
026      "Q0_OPERATING_INCOME",
027      "Q0_OPERATING_EXPENSES",
028      "Q0_EBITDA",
029  )
030  ID_COLUMN = "Id"
031  RANDOM_STATE = 42
032  VALIDATION_FRACTION = 0.20
033
034
035  def predictor_columns(train: pd.DataFrame) -> list[str]:
036      """Use the training schema only; exclude labels and Id by construction."""
037      required = set(TARGETS) | {ID_COLUMN}
038      missing = required.difference(train.columns)
039      if missing:
040          raise ValueError(f"Missing required training columns: {sorted(missing)}")
041      return [name for name in train.columns if name not in required]
042
043
044  def smape_by_target(y_true: pd.DataFrame, y_pred: pd.DataFrame) -> pd.Series:
045      """Rubric: 100/n * sum(|y-p| / (0.5*(|y|+|p|))) per target.
046
```

Code appendix | evaluation_framework.py

Data audit, train/validation split, mean baseline, exact sMAPE

```
047     The rubric leaves 0/0 undefined; use the standard convention of zero error
048     when both actual and prediction are zero. All other terms follow the formula.
049     """
050     if list(y_true.columns) != list(TARGETS) or list(y_pred.columns) != list(TARGETS):
051         raise ValueError("Both frames must contain the nine targets in rubric order")
052     if y_true.shape != y_pred.shape or not y_true.index.equals(y_pred.index):
053         raise ValueError("Truth and prediction must have identical shape and row index")
054     actual = y_true.to_numpy(dtype=np.float64)
055     predicted = y_pred.to_numpy(dtype=np.float64)
056     if not np.isfinite(actual).all() or not np.isfinite(predicted).all():
057         raise ValueError("sMAPE requires finite, non-missing targets and predictions")
058     denominator = 0.5 * (np.abs(actual) + np.abs(predicted))
059     terms = np.divide(
060         np.abs(actual - predicted),
061         denominator,
062         out=np.zeros_like(denominator),
063         where=denominator != 0,
064     )
065     return pd.Series(100.0 * terms.mean(axis=0), index=TARGETS, name="smape_percent")
066
067
068 def score_predictions(y_true: pd.DataFrame, y_pred: pd.DataFrame) -> tuple[pd.Series, float]:
069     """Shared scoring entry point for future models."""
070     per_target = smaпе_by_target(y_true, y_pred)
071     return per_target, float(per_target.mean())
072
073
074 def split_training_data(train: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame]:
075     """Rows, rather than columns, are partitioned; no test data is involved."""
076     return train_test_split(
077         train, test_size=VALIDATION_FRACTION, random_state=RANDOM_STATE, shuffle=True
078     )
079
080
081 def mean_baseline(train_part: pd.DataFrame, validation_part: pd.DataFrame) -> pd.DataFrame:
082     """Fit each constant from training labels only."""
083     means = train_part.loc[:, TARGETS].mean(axis=0)
084     return pd.DataFrame(
085         np.broadcast_to(means.to_numpy(dtype=np.float64), (len(validation_part), len(TARGETS))).copy(),
086         index=validation_part.index,
087         columns=TARGETS,
088     )
089
090
091 def column_audit(frame: pd.DataFrame) -> pd.DataFrame:
092     """Full column and missing-value inventory, including literal placeholders."""
```

Code appendix | evaluation_framework.py

Data audit, train/validation split, mean baseline, exact sMAPE

```
093     rows = []
094     for name in frame.columns:
095         series = frame[name]
096         missing = int(series.isna().sum())
097         nonfinite = (
098             int((~np.isfinite(series.to_numpy(dtype=np.float64)) & series.notna().to_numpy()).sum())
099             if pd.api.types.is_numeric_dtype(series)
100             else 0
101         )
102         placeholders = (
103             int(series.isin(["Missing", "Unknown"]).sum())
104             if pd.api.types.is_object_dtype(series)
105             else 0
106         )
107         rows.append(
108             {
109                 "column": name,
110                 "dtype": str(series.dtype),
111                 "kind": (
112                     "target" if name in TARGETS else "id" if name == ID_COLUMN
113                     else "categorical" if not pd.api.types.is_numeric_dtype(series)
114                     else "numeric"
115                 ),
116                 "missing_count": missing,
117                 "missing_pct": 100.0 * missing / len(frame),
118                 "nonfinite_count": nonfinite,
119                 "placeholder_count": placeholders,
120                 "unique_nonmissing": int(series.nunique(dropna=True)),
121             }
122         )
123     return pd.DataFrame(rows)
124
125
126 def target_audit(train: pd.DataFrame) -> pd.DataFrame:
127     rows = []
128     for name in TARGETS:
129         values = pd.to_numeric(train[name], errors="raise")
130         finite = values[np.isfinite(values)]
131         if finite.empty:
132             raise ValueError(f"Target {name} has no finite values")
133         rows.append(
134             {
135                 "target": name,
136                 "count": int(values.notna().sum()),
137                 "missing_count": int(values.isna().sum()),
138                 "nonfinite_count": int((~np.isfinite(values) & values.notna()).sum()),
```

Code appendix | evaluation_framework.py

Data audit, train/validation split, mean baseline, exact sMAPE

```
139         "negative_count": int((finite < 0).sum()),
140         "zero_count": int((finite == 0).sum()),
141         "min": float(finite.min()),
142         "p01": float(finite.quantile(0.01)),
143         "median": float(finite.median()),
144         "mean": float(finite.mean()),
145         "p99": float(finite.quantile(0.99)),
146         "max": float(finite.max()),
147     }
148 )
149 return pd.DataFrame(rows)
150
151
152 def fiscal_flag_audit(train: pd.DataFrame) -> pd.DataFrame:
153     """Check nominal binary quarter-end indicators for synthetic fractional values."""
154     rows = []
155     for quarter in range(11):
156         name = f"Q{quarter}_fiscal_year_end"
157         values = train[name].to_numpy(dtype=np.float64)
158         is_zero = np.isclose(values, 0)
159         is_one = np.isclose(values, 1)
160         rows.append({
161             "column": name,
162             "near_zero_count": int(is_zero.sum()),
163             "near_one_count": int(is_one.sum()),
164             "nonbinary_count": int((~is_zero & ~is_one).sum()),
165         })
166     return pd.DataFrame(rows)
167
168
169 def same_quarter_metadata_audit(train: pd.DataFrame) -> pd.DataFrame:
170     """Train-only diagnostic for metadata that resembles Q0 labels."""
171     pairs = [("totalRevenue", "Q0_REVENUES"), ("ebitda", "Q0_EBITDA")]
172     return pd.DataFrame([
173         {
174             "predictor": predictor,
175             "target": target,
176             "pearson_correlation": float(train[predictor].corr(train[target])),
177             "exact_match_count": int(train[predictor].eq(train[target]).sum()),
178         }
179         for predictor, target in pairs
180     ])
181
182
183 def main() -> None:
184     OUTPUT.mkdir(exist_ok=True)
```

Code appendix | evaluation_framework.py

Data audit, train/validation split, mean baseline, exact sMAPE

```
185 train = pd.read_csv(ROOT / "train.csv", low_memory=False)
186 features = predictor_columns(train)
187 if len(features) != len(set(features)) or not train.columns.is_unique:
188     raise ValueError("Duplicate training column names")
189
190 train_columns = column_audit(train)
191 target_stats = target_audit(train)
192 train_columns.to_csv(OUTPUT / "train_columns.csv", index=False)
193 target_stats.to_csv(OUTPUT / "target_distributions.csv", index=False)
194 fiscal_flags = fiscal_flag_audit(train)
195 fiscal_flags.to_csv(OUTPUT / "fiscal_flags.csv", index=False)
196 same_quarter_metadata_audit(train).to_csv(OUTPUT / "same_quarter_metadata.csv", index=False)
197
198 train_part, validation_part = split_training_data(train)
199 # These variables document the exact model interface for later estimators.
200 X_train, X_valid = train_part[features], validation_part[features]
201 y_train, y_valid = train_part[list(TARGETS)], validation_part[list(TARGETS)]
202 if y_train.isna().any().any() or y_valid.isna().any().any():
203     raise ValueError("Targets contain missing values; decide a label policy before scoring")
204 predictions = mean_baseline(train_part, validation_part)
205 scores, average = score_predictions(y_valid, predictions)
206 split_labels = pd.Series("train", index=train.index, name="split")
207 split_labels.loc[validation_part.index] = "validation"
208 pd.DataFrame({ID_COLUMN: train[ID_COLUMN], "split": split_labels}).to_csv(
209     OUTPUT / "split_ids.csv", index=False
210 )
211 baseline = pd.DataFrame(
212     {"target": TARGETS, "training_mean": y_train.mean().to_numpy(),
213      "validation_smape_percent": scores.to_numpy()}
214 )
215 baseline.to_csv(OUTPUT / "baseline_scores.csv", index=False)
216
217 # Read the held-out file only after the training-only split and baseline are fixed.
218 # It is used exclusively for schema/missingness checks, never for fitting/tuning.
219 test = pd.read_csv(ROOT / "test_individual.csv", low_memory=False)
220 test_columns = column_audit(test)
221 test_columns.to_csv(OUTPUT / "test_columns.csv", index=False)
222 test_features = [name for name in test.columns if name != ID_COLUMN]
223 missing_from_test = sorted(set(features) - set(test_features))
224 extra_in_test = sorted(set(test_features) - set(features))
225 same_order = features == test_features
226
227 categorical = [name for name in features if not pd.api.types.is_numeric_dtype(train[name])]
228 numeric = [name for name in features if pd.api.types.is_numeric_dtype(train[name])]
229 fiscal_columns = [name for name in features if name.endswith("_fiscal_year_end")]
230 historical = [name for name in features if name.startswith(tuple(f"Q{i}_" for i in range(1, 11)))]
```

Code appendix | evaluation_framework.py

Data audit, train/validation split, mean baseline, exact sMAPE

```
231 metadata = [name for name in features if name not in historical + fiscal_columns]
232 summary = {
233     "train_shape": list(train.shape),
234     "test_shape": list(test.shape),
235     "target_columns": list(TARGETS),
236     "predictor_count": len(features),
237     "numeric_predictors": numeric,
238     "categorical_predictors": categorical,
239     "train_test_predictors_match": not missing_from_test and not extra_in_test,
240     "train_test_predictor_order_matches": same_order,
241     "missing_from_test": missing_from_test,
242     "extra_in_test": extra_in_test,
243     "id_train_unique": bool(train[ID_COLUMN].is_unique),
244     "id_test_unique": bool(test[ID_COLUMN].is_unique),
245     "id_overlap_count": int(len(set(train[ID_COLUMN]) & set(test[ID_COLUMN]))),
246     "duplicate_train_predictor_rows": int(train[features].duplicated().sum()),
247     "duplicate_train_historical_rows": int(train[historical].duplicated().sum()),
248     "train_missing_cells": int(train.isna().sum().sum()),
249     "test_missing_cells": int(test.isna().sum().sum()),
250     "train_nonfinite_numeric_cells": int(train.columns.nonfinite_count.sum()),
251     "test_nonfinite_numeric_cells": int(test.columns.nonfinite_count.sum()),
252     "train_nonbinary_fiscal_cells": int(fiscal_flags.nonbinary_count.sum()),
253     "validation_fraction": VALIDATION_FRACTION,
254     "random_state": RANDOM_STATE,
255     "train_split_rows": len(X_train),
256     "validation_split_rows": len(X_valid),
257     "baseline_mean_smape_percent": average,
258     "q0_metadata_columns_for_leakage_review": metadata,
259 }
260 (OUTPUT / "summary.json").write_text(json.dumps(summary, indent=2, ensure_ascii=False) + "\n")
261 print(json.dumps({
262     "train_shape": summary["train_shape"],
263     "test_shape": summary["test_shape"],
264     "predictor_count": summary["predictor_count"],
265     "categorical_predictors": categorical,
266     "predictors_match": summary["train_test_predictors_match"],
267     "baseline_mean_smape_percent": average,
268 }, indent=2, ensure_ascii=False))
269
270
271 if __name__ == "__main__":
272     main()
```

Code appendix | benchmark_models.py

Common-split non-DL models and feedforward neural network

```
001  """Train-only, single-split regression benchmark. Never reads test_individual.csv.
002
003  Run all models in separate processes (limits peak memory):
004      python benchmark_models.py --all
005  Or run one model, then rebuild the comparison:
006      python benchmark_models.py --model ridge
007      python benchmark_models.py --summarize
008  """
009
010  from __future__ import annotations
011
012  import argparse
013  import json
014  import subprocess
015  import sys
016  import time
017  from pathlib import Path
018
019  import numpy as np
020  import pandas as pd
021  from sklearn.compose import ColumnTransformer
022  from sklearn.ensemble import ExtraTreesRegressor, HistGradientBoostingRegressor, RandomForestRegressor
023  from sklearn.impute import SimpleImputer
024  from sklearn.linear_model import Ridge
025  from sklearn.pipeline import Pipeline
026  from sklearn.preprocessing import FunctionTransformer, OneHotEncoder, StandardScaler
027
028  from evaluation_framework import (
029      ID_COLUMN,
030      RANDOM_STATE,
031      ROOT,
032      TARGETS,
033      mean_baseline,
034      predictor_columns,
035      score_predictions,
036      split_training_data,
037  )
038
039  OUTPUT = ROOT / "benchmark_outputs"
040  PREDICTIONS = OUTPUT / "validation_predictions"
041  MODELS = ("mean", "ridge", "random_forest", "extra_trees", "hist_gradient_boosting", "lightgbm", "neural_network")
042  TREE_THREADS = 4
043
044
```

Code appendix | benchmark_models.py

Common-split non-DL models and feedforward neural network

```
045 def data_split() -> tuple[pd.DataFrame, pd.DataFrame, list[str], list[str], list[str]]:
046     train = pd.read_csv(ROOT / "train.csv", low_memory=False)
047     features = predictor_columns(train)
048     numeric = [c for c in features if pd.api.types.is_numeric_dtype(train[c])]
049     categorical = [c for c in features if c not in numeric]
050     train_part, valid_part = split_training_data(train)
051     expected = ROOT / "audit_outputs" / "split_ids.csv"
052     if expected.exists():
053         previous = pd.read_csv(expected)
054         valid_ids = previous.loc[previous["split"] == "validation", ID_COLUMN].to_numpy()
055         if len(valid_ids) != len(valid_part) or set(valid_ids) != set(valid_part[ID_COLUMN]):
056             raise ValueError("Validation IDs differ from the existing audit split")
057     if ID_COLUMN in features or set(TARGETS).intersection(features):
058         raise AssertionError("Id or target leaked into predictors")
059     return train_part, valid_part, features, numeric, categorical
060
061
062 def preprocessor(numeric: list[str], categorical: list[str], *, scaled: bool) -> ColumnTransformer:
063     numeric_steps: list[tuple[str, object]] = [("impute", SimpleImputer(strategy="median"))]
064     if scaled:
065         numeric_steps += [
066             ("signed_log", FunctionTransformer(np.arcsinh, validate=False)),
067             ("scale", StandardScaler()),
068         ]
069     categorical_steps = [
070         ("impute", SimpleImputer(strategy="most_frequent")),
071         ("one_hot", OneHotEncoder(handle_unknown="ignore", sparse_output=False, dtype=np.float32)),
072     ]
073     return ColumnTransformer(
074         [("numeric", Pipeline(numeric_steps), numeric),
075          ("categorical", Pipeline(categorical_steps), categorical)],
076         sparse_threshold=0.0,
077     )
078
079
080 def matrices(
081     train_part: pd.DataFrame,
082     valid_part: pd.DataFrame,
083     features: list[str],
084     numeric: list[str],
085     categorical: list[str],
086     *,
087     scaled: bool,
088 ) -> tuple[np.ndarray, np.ndarray]:
```

Code appendix | benchmark_models.py

Common-split non-DL models and feedforward neural network

```
089     processor = preprocessor(numeric, categorical, scaled=scaled)
090     x_train = processor.fit_transform(train_part[features]).astype(np.float32, copy=False)
091     x_valid = processor.transform(valid_part[features]).astype(np.float32, copy=False)
092     if not np.isfinite(x_train).all() or not np.isfinite(x_valid).all():
093         raise ValueError("Preprocessed features contain non-finite values")
094     return x_train, x_valid
095
096
097 def train_neural_network(
098     x_train: np.ndarray, y_train: np.ndarray, x_valid: np.ndarray
099 ) -> tuple[np.ndarray, float, float]:
100     """Small 2-layer network; transform and scaling statistics come from train only."""
101     import torch
102     from torch import nn
103     from torch.utils.data import DataLoader, TensorDataset
104
105     fit_start = time.perf_counter()
106     torch.manual_seed(RANDOM_STATE)
107     torch.set_num_threads(TREE_THREADS)
108     scale = np.median(np.abs(y_train), axis=0)
109     scale = np.where(scale > 0, scale, 1.0)
110     transformed = np.arcsinh(y_train / scale)
111     center = transformed.mean(axis=0)
112     spread = transformed.std(axis=0)
113     spread = np.where(spread > 0, spread, 1.0)
114     z_train = ((transformed - center) / spread).astype(np.float32)
115
116     network = nn.Sequential(
117         nn.Linear(x_train.shape[1], 128), nn.ReLU(),
118         nn.Linear(128, 64), nn.ReLU(),
119         nn.Linear(64, len(TARGETS)),
120     )
121     optimizer = torch.optim.AdamW(network.parameters(), lr=1e-3, weight_decay=1e-4)
122     loss_fn = nn.SmoothL1Loss()
123     loader = DataLoader(
124         TensorDataset(torch.from_numpy(x_train), torch.from_numpy(z_train)),
125         batch_size=2048, shuffle=True, num_workers=0,
126         generator=torch.Generator().manual_seed(RANDOM_STATE),
127     )
128     network.train()
129     for epoch in range(12):
130         total_loss = 0.0
131         for x_batch, y_batch in loader:
132             optimizer.zero_grad(set_to_none=True)
```

Code appendix | benchmark_models.py

Common-split non-DL models and feedforward neural network

```
133         prediction = network(x_batch)
134         loss = loss_fn(prediction, y_batch)
135         loss.backward()
136         optimizer.step()
137         total_loss += loss.item() * len(x_batch)
138         print(f"neural_network epoch {epoch + 1:02d}/12 loss={total_loss / len(x_train):.5f}", flush=True)
139     fit_seconds = time.perf_counter() - fit_start
140     prediction_start = time.perf_counter()
141     network.eval()
142     chunks = []
143     with torch.no_grad():
144         for start in range(0, len(x_valid), 4096):
145             chunks.append(network(torch.from_numpy(x_valid[start:start + 4096])).numpy())
146     z_pred = np.vstack(chunks).astype(np.float64)
147     values = np.sinh(z_pred * spread + center) * scale
148     return values, fit_seconds, time.perf_counter() - prediction_start
149
150
151 def fit_model(
152     name: str,
153     train_part: pd.DataFrame,
154     valid_part: pd.DataFrame,
155     features: list[str],
156     numeric: list[str],
157     categorical: list[str],
158 ) -> tuple[pd.DataFrame, float, float, dict]:
159     """Training time includes train-fold preprocessing and fit, not CSV loading."""
160     y_train = train_part[list(TARGETS)].to_numpy(dtype=np.float64)
161     start = time.perf_counter()
162     config: dict = {}
163     if name == "mean":
164         pred = mean_baseline(train_part, valid_part)
165         training_seconds = time.perf_counter() - start
166         return pred, training_seconds, 0.0, {"estimator": "train-fold target mean"}
167
168     scaled = name in ("ridge", "neural_network")
169     x_train, x_valid = matrices(
170         train_part, valid_part, features, numeric, categorical, scaled=scaled
171     )
172     preprocessing_seconds = time.perf_counter() - start
173     config["preprocessing"] = "median imputation, one-hot categories" + (
174         ", arcsinh and standard scaling of numeric features" if scaled else ""
175     )
176     config["encoded_feature_count"] = int(x_train.shape[1])
```

Code appendix | benchmark_models.py

Common-split non-DL models and feedforward neural network

```
177
178     if name == "neural_network":
179         config.update({"architecture": [128, 64], "epochs": 12, "batch_size": 2048,
180                        "loss": "SmoothL1", "target_transform": "arcsinh(y / median_abs_train)"})
181         values, fit_seconds, predict_seconds = train_neural_network(x_train, y_train, x_valid)
182         training_seconds = preprocessing_seconds + fit_seconds
183     else:
184         if name == "ridge":
185             model = Ridge(alpha=100.0, solver="lsqr")
186             config.update({"alpha": 100.0, "solver": "lsqr"})
187         elif name == "random_forest":
188             model = RandomForestRegressor(
189                 n_estimators=48, max_depth=14, min_samples_leaf=8,
190                 max_features=0.7, n_jobs=TREE_THREADS, random_state=RANDOM_STATE,
191             )
192             config.update({"n_estimators": 48, "max_depth": 14,
193                           "min_samples_leaf": 8, "max_features": 0.7})
194         elif name == "extra_trees":
195             model = ExtraTreesRegressor(
196                 n_estimators=48, max_depth=14, min_samples_leaf=8,
197                 max_features=0.7, n_jobs=TREE_THREADS, random_state=RANDOM_STATE,
198             )
199             config.update({"n_estimators": 48, "max_depth": 14,
200                           "min_samples_leaf": 8, "max_features": 0.7})
201         elif name == "hist_gradient_boosting":
202             model = None
203             config.update({"max_iter": 80, "max_leaf_nodes": 31,
204                           "l2_regularization": 1.0, "target_models": 9})
205         elif name == "lightgbm":
206             from lightgbm import LGBMRegressor
207
208             model = None
209             config.update({"n_estimators": 120, "learning_rate": 0.08,
210                           "num_leaves": 31, "min_child_samples": 40,
211                           "target_models": 9})
212         else:
213             raise ValueError(f"Unknown model: {name}")
214
215     if name in ("hist_gradient_boosting", "lightgbm"):
216         fitted = []
217         for j, target in enumerate(TARGETS):
218             if name == "hist_gradient_boosting":
219                 estimator = HistGradientBoostingRegressor(
220                     max_iter=80, max_leaf_nodes=31, max_bins=128,
```

Code appendix | benchmark_models.py

Common-split non-DL models and feedforward neural network

```
221         l2_regularization=1.0, early_stopping=False,
222         random_state=RANDOM_STATE,
223     )
224     else:
225         estimator = LGBMRegressor(
226             n_estimators=120, learning_rate=0.08, num_leaves=31,
227             min_child_samples=40, colsample_bytree=0.8,
228             n_jobs=TREE_THREADS, random_state=RANDOM_STATE,
229             deterministic=True, force_col_wise=True, verbosity=-1,
230         )
231         estimator.fit(x_train, y_train[:, j])
232         fitted.append(estimator)
233         print(f"{name}: fitted {target} ({j + 1}/9)", flush=True)
234         training_seconds = time.perf_counter() - start
235         predict_start = time.perf_counter()
236         values = np.column_stack([est.predict(x_valid) for est in fitted])
237         predict_seconds = time.perf_counter() - predict_start
238     else:
239         model.fit(x_train, y_train)
240         training_seconds = time.perf_counter() - start
241         predict_start = time.perf_counter()
242         values = model.predict(x_valid)
243         predict_seconds = time.perf_counter() - predict_start
244
245     prediction = pd.DataFrame(np.asarray(values, dtype=np.float64),
246                             index=valid_part.index, columns=TARGETS)
247     return prediction, training_seconds, predict_seconds, config
248
249
250 def diagnostics(train_part: pd.DataFrame, prediction: pd.DataFrame) -> dict:
251     """Descriptive range checks; no validation-driven clipping or fitting."""
252     result = {}
253     for target in TARGETS:
254         truth = train_part[target].to_numpy(dtype=np.float64)
255         pred = prediction[target].to_numpy(dtype=np.float64)
256         scale = float(np.quantile(np.abs(truth), 0.99))
257         result[target] = {
258             "prediction_min": float(pred.min()),
259             "prediction_max": float(pred.max()),
260             "prediction_median": float(np.median(pred)),
261             "outside_training_range_count": int(((pred < truth.min()) | (pred > truth.max())).sum()),
262             "above_10x_training_abs_p99_count": int((np.abs(pred) > 10 * scale).sum()),
263             "negative_prediction_count": int((pred < 0).sum()),
264         }
```

Code appendix | benchmark_models.py

Common-split non-DL models and feedforward neural network

```
265     return result
266
267
268 def run_model(name: str) -> None:
269     if name not in MODELS:
270         raise ValueError(name)
271     PREDICTIONS.mkdir(parents=True, exist_ok=True)
272     train_part, valid_part, features, numeric, categorical = data_split()
273     prediction, train_seconds, pred_seconds, config = fit_model(
274         name, train_part, valid_part, features, numeric, categorical
275     )
276     y_valid = valid_part[list(TARGETS)]
277     per_target, average = score_predictions(y_valid, prediction)
278
279     # Id is recorded solely as an alignment key, never used as a predictor.
280     aligned = pd.DataFrame({"row_index": valid_part.index.to_numpy(),
281                            ID_COLUMN: valid_part[ID_COLUMN].to_numpy()}, index=valid_part.index)
282     if not (OUTPUT / "validation_truth.csv").exists():
283         aligned.join(y_valid).to_csv(OUTPUT / "validation_truth.csv", index=False,
284                                     float_format="%.17g")
285     else:
286         previous = pd.read_csv(OUTPUT / "validation_truth.csv", usecols=["row_index", ID_COLUMN])
287         if not np.array_equal(previous["row_index"], aligned["row_index"]) or not np.array_equal(
288             previous[ID_COLUMN], aligned[ID_COLUMN]
289         ):
290             raise ValueError("Saved validation truth uses a different row order")
291     aligned.join(prediction).to_csv(PREDICTIONS / f"{name}.csv", index=False,
292                                   float_format="%.17g")
293     metrics = {
294         "model": name,
295         "training_seconds": train_seconds,
296         "prediction_seconds": pred_seconds,
297         "mean_smape_percent": average,
298         "target_smape_percent": {t: float(per_target[t]) for t in TARGETS},
299         "validation_rows": len(valid_part),
300         "random_state": RANDOM_STATE,
301         "config": config,
302         "range_diagnostics": diagnostics(train_part, prediction),
303     }
304     (OUTPUT / f"{name}_metrics.json").write_text(json.dumps(metrics, indent=2) + "\n")
305     print(f"{name}: mean SMAPE={average:.4f}%, training={train_seconds:.1f}s, "
306           f"prediction={pred_seconds:.1f}s", flush=True)
307
308
```

Code appendix | benchmark_models.py

Common-split non-DL models and feedforward neural network

```
309 def summarize() -> None:
310     files = [OUTPUT / f"{name}_metrics.json" for name in MODELS]
311     missing = [path.name for path in files if not path.exists()]
312     if missing:
313         raise FileNotFoundError(f"Missing model results: {missing}")
314     metrics = [json.loads(path.read_text()) for path in files]
315     baseline = next(m["mean_smape_percent"] for m in metrics if m["model"] == "mean")
316     rows = []
317     for m in metrics:
318         row = {
319             "model": m["model"],
320             "mean_smape_percent": m["mean_smape_percent"],
321             "improvement_vs_mean_percent": 100.0 * (baseline - m["mean_smape_percent"]) / baseline,
322             "training_seconds": m["training_seconds"],
323             "prediction_seconds": m["prediction_seconds"],
324         }
325         row.update(m["target_smape_percent"])
326         rows.append(row)
327     comparison = pd.DataFrame(rows).sort_values("mean_smape_percent", kind="stable")
328     comparison.to_csv(OUTPUT / "comparison.csv", index=False)
329     print(comparison[["model", "mean_smape_percent", "improvement_vs_mean_percent",
330                     "training_seconds"]].to_string(index=False, float_format=lambda v: f"{v:.3f}"))
331
332
333 def main() -> None:
334     parser = argparse.ArgumentParser()
335     group = parser.add_mutually_exclusive_group(required=True)
336     group.add_argument("--model", choices=MODELS)
337     group.add_argument("--all", action="store_true")
338     group.add_argument("--summarize", action="store_true")
339     args = parser.parse_args()
340     if args.all:
341         for name in MODELS:
342             subprocess.run([sys.executable, str(Path(__file__).resolve()), "--model", name], check=True)
343         summarize()
344     elif args.summarize:
345         summarize()
346     else:
347         run_model(args.model)
348
349
350 if __name__ == "__main__":
351     main()
```

Code appendix | oof_models.py

Five-fold tree-family OOF predictions

```
001  """Five-fold OOF predictions for the three selected tree families.
002
003  Only train.csv is read. Run `python oof_models.py --all`, or one model at a time
004  with `--model random_forest|lightgbm|hist_gradient_boosting`.
005  """
006
007  from __future__ import annotations
008
009  import argparse
010  import gc
011  import hashlib
012  import json
013  import os
014  import subprocess
015  import sys
016  import time
017  from pathlib import Path
018
019  os.environ.setdefault("LOKY_MAX_CPU_COUNT", "4")
020
021  import numpy as np
022  import pandas as pd
023  from sklearn.ensemble import HistGradientBoostingRegressor, RandomForestRegressor
024  from sklearn.model_selection import KFold
025
026  from benchmark_models import TREE_THREADS, matrices
027  from evaluation_framework import ID_COLUMN, RANDOM_STATE, ROOT, TARGETS, predictor_columns, score_predictions
028
029  OUTPUT = ROOT / "oof_outputs"
030  CACHE = OUTPUT / "fold_cache"
031  MODELS = ("random_forest", "lightgbm", "hist_gradient_boosting")
032  N_SPLITS = 5
033  PREPROCESS_VERSION = 1
034
035
036  def model_config(name: str) -> dict:
037      shared = {"folds": N_SPLITS, "shuffle": True, "random_state": RANDOM_STATE,
038               "preprocess_version": PREPROCESS_VERSION,
039               "preprocessing": "train-fold median imputation + one-hot categories"}
040      if name == "random_forest":
041          return shared | {"n_estimators": 48, "max_depth": 14, "min_samples_leaf": 8,
042                          "max_features": 0.7, "n_jobs": TREE_THREADS}
043      if name == "lightgbm":
044          return shared | {"n_estimators": 120, "learning_rate": 0.08,
045                          "num_leaves": 31, "min_child_samples": 40,
046                          "colsample_bytree": 0.8, "n_jobs": TREE_THREADS}
047      if name == "hist_gradient_boosting":
048          return shared | {"max_iter": 80, "max_leaf_nodes": 31, "max_bins": 128,
049                          "l2_regularization": 1.0, "early_stopping": False}
050      raise ValueError(name)
```

Code appendix | oof_models.py

Five-fold tree-family OOF predictions

```
051
052
053 def configuration_tag(name: str) -> str:
054     payload = json.dumps(model_config(name), sort_keys=True).encode()
055     return hashlib.sha256(payload).hexdigest()[:12]
056
057
058 def train_predict_fold(
059     name: str, x_train: np.ndarray, y_train: np.ndarray, x_valid: np.ndarray,
060     fold: int,
061 ) -> np.ndarray:
062     if name == "random_forest":
063         estimator = RandomForestRegressor(
064             n_estimators=48, max_depth=14, min_samples_leaf=8,
065             max_features=0.7, n_jobs=TREE_THREADS, random_state=RANDOM_STATE + fold,
066         )
067         estimator.fit(x_train, y_train)
068         values = estimator.predict(x_valid)
069         del estimator
070         return np.asarray(values, dtype=np.float64)
071
072     values = np.empty((len(x_valid), len(TARGETS)), dtype=np.float64)
073     if name == "lightgbm":
074         from lightgbm import LGBMRegressor
075     for j, target in enumerate(TARGETS):
076         if name == "lightgbm":
077             estimator = LGBMRegressor(
078                 n_estimators=120, learning_rate=0.08, num_leaves=31,
079                 min_child_samples=40, colsample_bytree=0.8,
080                 n_jobs=TREE_THREADS, random_state=RANDOM_STATE + fold,
081                 deterministic=True, force_col_wise=True, verbosity=-1,
082             )
083         else:
084             estimator = HistGradientBoostingRegressor(
085                 max_iter=80, max_leaf_nodes=31, max_bins=128,
086                 l2_regularization=1.0, early_stopping=False,
087                 random_state=RANDOM_STATE + fold,
088             )
089         estimator.fit(x_train, y_train[:, j])
090         values[:, j] = estimator.predict(x_valid)
091         del estimator
092         print(f"{name} fold {fold + 1}/{N_SPLITS}: {target} done", flush=True)
093     return values
094
095
096 def run_model(name: str, *, force: bool = False) -> None:
097     if name not in MODELS:
098         raise ValueError(name)
099     CACHE.mkdir(parents=True, exist_ok=True)
100     train = pd.read_csv(ROOT / "train.csv", low_memory=False)
```

Code appendix | oof_models.py

Five-fold tree-family OOF predictions

```
101     features = predictor_columns(train)
102     numeric = [c for c in features if pd.api.types.is_numeric_dtype(train[c])]
103     categorical = [c for c in features if c not in numeric]
104     if ID_COLUMN in features or set(TARGETS) & set(features):
105         raise AssertionError("Id or targets entered the feature set")
106     truth = train[list(TARGETS)]
107     splitter = KFold(n_splits=N_SPLITS, shuffle=True, random_state=RANDOM_STATE)
108     fold_by_row = np.full(len(train), -1, dtype=np.int8)
109     for fold, (_, valid_idx) in enumerate(splitter.split(train)):
110         fold_by_row[valid_idx] = fold
111     if (fold_by_row < 0).any():
112         raise AssertionError("A training row has no validation fold")
113     assignment = pd.DataFrame({"row_index": np.arange(len(train)),
114                               ID_COLUMN: train[ID_COLUMN].to_numpy(),
115                               "fold": fold_by_row})
116     assignment_path = OUTPUT / "fold_assignments.parquet"
117     if assignment_path.exists():
118         existing = pd.read_parquet(assignment_path)
119         if not existing.equals(assignment):
120             raise ValueError("Existing fold assignments do not match")
121     else:
122         assignment.to_parquet(assignment_path, index=False, compression="zstd")
123     output = np.full((len(train), len(TARGETS)), np.nan, dtype=np.float64)
124     fold_times = []
125     tag = configuration_tag(name)
126
127     for fold, (fit_idx, valid_idx) in enumerate(splitter.split(train)):
128         fold_path = CACHE / f"{name}_{tag}_fold{fold + 1}.parquet"
129         if fold_path.exists() and not force:
130             cached = pd.read_parquet(fold_path)
131             if not np.array_equal(cached["row_index"].to_numpy(), valid_idx) or not np.array_equal(
132                 cached[ID_COLUMN].to_numpy(), train.iloc[valid_idx][ID_COLUMN].to_numpy()
133             ):
134                 raise ValueError(f"Cached fold alignment mismatch: {fold_path}")
135             values = cached[list(TARGETS)].to_numpy(dtype=np.float64)
136             seconds = float(cached["fold_seconds"].iloc[0])
137             print(f"{name} fold {fold + 1}/{N_SPLITS}: loaded checkpoint", flush=True)
138         else:
139             started = time.perf_counter()
140             fit_rows, valid_rows = train.iloc[fit_idx], train.iloc[valid_idx]
141             x_train, x_valid = matrices(
142                 fit_rows, valid_rows, features, numeric, categorical, scaled=False
143             )
144             y_train = fit_rows[list(TARGETS)].to_numpy(dtype=np.float64)
145             values = train_predict_fold(name, x_train, y_train, x_valid, fold)
146             seconds = time.perf_counter() - started
147             if values.shape != (len(valid_idx), len(TARGETS)) or not np.isfinite(values).all():
148                 raise ValueError(f"Invalid OOF predictions for {name}, fold {fold + 1}")
149             cached = pd.DataFrame(values, columns=TARGETS)
150             cached.insert(0, ID_COLUMN, train.iloc[valid_idx][ID_COLUMN].to_numpy())
```

Code appendix | oof_models.py

Five-fold tree-family OOF predictions

```
151         cached.insert(0, "row_index", valid_idx)
152         cached["fold_seconds"] = seconds
153         cached.to_parquet(fold_path, index=False, compression="zstd")
154         print(f"{name} fold {fold + 1}/{N_SPLITS}: {seconds:.1f}s", flush=True)
155         del fit_rows, valid_rows, x_train, x_valid, y_train, cached
156         gc.collect()
157         output[valid_idx] = values
158         fold_times.append(seconds)
159
160     if not np.isfinite(output).all():
161         raise ValueError("OOF matrix is incomplete or non-finite")
162     prediction = pd.DataFrame(output, columns=TARGETS)
163     per_target, mean = score_predictions(truth, prediction)
164     aligned = prediction.copy()
165     aligned.insert(0, ID_COLUMN, train[ID_COLUMN].to_numpy())
166     aligned.insert(0, "row_index", np.arange(len(train)))
167     aligned.to_parquet(OUTPUT / f"{name}_oof.parquet", index=False, compression="zstd")
168     metrics = {
169         "model": name, "n_rows": len(train), "n_splits": N_SPLITS,
170         "random_state": RANDOM_STATE, "config": model_config(name),
171         "fold_seconds": fold_times, "total_fold_seconds": float(sum(fold_times)),
172         "mean_smape_percent": mean,
173         "target_smape_percent": {t: float(per_target[t]) for t in TARGETS},
174     }
175     (OUTPUT / f"{name}_metrics.json").write_text(json.dumps(metrics, indent=2) + "\n")
176     print(f"{name} OOF mean SMAPE={mean:.5f}%", flush=True)
177
178
179 def main() -> None:
180     parser = argparse.ArgumentParser()
181     group = parser.add_mutually_exclusive_group(required=True)
182     group.add_argument("--model", choices=MODELS)
183     group.add_argument("--all", action="store_true")
184     parser.add_argument("--force", action="store_true", help="Refit even if fold checkpoints exist")
185     args = parser.parse_args()
186     if args.all:
187         for name in MODELS:
188             command = [sys.executable, str(Path(__file__).resolve()), "--model", name]
189             if args.force:
190                 command.append("--force")
191             subprocess.run(command, check=True)
192     else:
193         run_model(args.model, force=args.force)
194
195
196 if __name__ == "__main__":
197     main()
```

Code appendix | hard_target_oof.py

Fixed lag features and target-transform experiments

```
001  """Controlled 5-fold LightGBM experiments for three difficult targets.
002
003  Both variants use the same engineered lag features and folds. The only change
004  between them is raw versus train-fold arcsinh target values. No test file used.
005  """
006
007  from __future__ import annotations
008
009  import argparse
010  import gc
011  import hashlib
012  import json
013  import os
014  import subprocess
015  import sys
016  import time
017  from pathlib import Path
018
019  os.environ.setdefault("MPLCONFIGDIR", "/private/tmp/matplotlib-codex-financial")
020
021  import numpy as np
022  import pandas as pd
023  from sklearn.model_selection import KFold
024
025  from benchmark_models import TREE_THREADS, matrices
026  from evaluation_framework import ID_COLUMN, RANDOM_STATE, ROOT, TARGETS, predictor_columns, score_predictions
027
028  OUTPUT = ROOT / "oof_outputs"
029  CACHE = OUTPUT / "fold_cache"
030  HARD_TARGETS = ("Q0_OPERATING_INCOME", "Q0_EBITDA", "Q0_GROSS_PROFIT")
031  N_SPLITS = 5
032  CONFIG = {"n_estimators": 120, "learning_rate": 0.08, "num_leaves": 31,
033           "min_child_samples": 40, "colsample_bytree": 0.8, "threads": TREE_THREADS,
034           "n_splits": 5, "random_state": RANDOM_STATE, "lag_feature_version": 1,
035           "preprocessing": "fit-fold median imputation + one-hot categories"}
036  TAG = hashlib.sha256(json.dumps(CONFIG, sort_keys=True).encode()).hexdigest()[:12]
037
038
039  def lag_features(train: pd.DataFrame, target: str) -> pd.DataFrame:
040      """Row-wise Q1-Q10 features; positive slope means growth toward Q1."""
041      suffix = target.removeprefix("Q0_")
042      values = np.column_stack([train[f"Q{k}_{suffix}"].to_numpy(dtype=np.float64)
043                               for k in range(1, 11)])
044      if not np.isfinite(values).all():
045          raise ValueError(f"Lag values for {target} are not finite")
046      recent = values[:, :4]
047      weights4 = np.arange(3, -1, -1, dtype=np.float64) - 1.5
```

Code appendix | hard_target_oof.py

Fixed lag features and target-transform experiments

```
048     weights10 = np.arange(9, -1, -1, dtype=np.float64) - 4.5
049     stem = target.lower()
050     return pd.DataFrame({
051         f"{stem}_lag_q1": values[:, 0],
052         f"{stem}_q1_minus_q2": values[:, 0] - values[:, 1],
053         f"{stem}_q1_sign": np.sign(values[:, 0]),
054         f"{stem}_q2_sign": np.sign(values[:, 1]),
055         f"{stem}_q1_q2_same_sign": np.sign(values[:, 0]) * np.sign(values[:, 1]),
056         f"{stem}_q1_q4_mean": recent.mean(axis=1),
057         f"{stem}_q1_q4_median": np.median(recent, axis=1),
058         f"{stem}_q1_q4_std": recent.std(axis=1),
059         f"{stem}_q1_q4_slope_recent": recent @ weights4 / np.dot(weights4, weights4),
060         f"{stem}_q1_q10_mean": values.mean(axis=1),
061         f"{stem}_q1_q10_slope_recent": values @ weights10 / np.dot(weights10, weights10),
062     }, index=train.index)
063
064
065 def fit_lightgbm(x_train: np.ndarray, y_train: np.ndarray, x_valid: np.ndarray,
066                 *, fold: int, transform: bool) -> np.ndarray:
067     from lightgbm import LGBMRegressor
068
069     if transform:
070         scale = float(np.median(np.abs(y_train)))
071         if scale <= 0:
072             scale = 1.0
073         fitted_target = np.arcsinh(y_train / scale)
074     else:
075         scale = 1.0
076         fitted_target = y_train
077     model = LGBMRegressor(
078         n_estimators=120, learning_rate=0.08, num_leaves=31,
079         min_child_samples=40, colsample_bytree=0.8,
080         n_jobs=TREE_THREADS, random_state=RANDOM_STATE + fold,
081         deterministic=True, force_col_wise=True, verbosity=-1,
082     )
083     model.fit(x_train, fitted_target)
084     predicted = model.predict(x_valid)
085     del model
086     if transform:
087         predicted = np.sinh(predicted) * scale
088     if not np.isfinite(predicted).all():
089         raise ValueError("Non-finite predictions from target inverse transformation")
090     return np.asarray(predicted, dtype=np.float64)
091
092
093 def target_score(truth: pd.DataFrame, target: str, prediction: np.ndarray) -> float:
094     surrogate = truth.copy()
```

Code appendix | hard_target_oof.py

Fixed lag features and target-transform experiments

```
095     surrogate[target] = prediction
096     per_target, _ = score_predictions(truth, surrogate)
097     return float(per_target[target])
098
099
100 def run_target(target: str, *, force: bool = False) -> None:
101     if target not in HARD_TARGETS:
102         raise ValueError(target)
103     CACHE.mkdir(parents=True, exist_ok=True)
104     train = pd.read_csv(ROOT / "train.csv", low_memory=False)
105     base = predictor_columns(train)
106     engineered = lag_features(train, target)
107     train = pd.concat([train, engineered], axis=1)
108     features = base + list(engineered.columns)
109     numeric = [c for c in features if pd.api.types.is_numeric_dtype(train[c])]
110     categorical = [c for c in features if c not in numeric]
111     if ID_COLUMN in features or set(TARGETS) & set(features):
112         raise AssertionError("Id or Q0 target entered predictors")
113     output = np.full((len(train), 2), np.nan, dtype=np.float64)
114     fold_times = []
115     splitter = KFold(n_splits=N_SPLITS, shuffle=True, random_state=RANDOM_STATE)
116     for fold, (fit_idx, valid_idx) in enumerate(splitter.split(train)):
117         path = CACHE / f"hard_{target}_{TAG}_fold{fold + 1}.parquet"
118         if path.exists() and not force:
119             cached = pd.read_parquet(path)
120             if not np.array_equal(cached.row_index.to_numpy(), valid_idx) or not np.array_equal(
121                 cached[ID_COLUMN].to_numpy(), train.iloc[valid_idx][ID_COLUMN].to_numpy()
122             ):
123                 raise ValueError("Hard-target fold cache alignment mismatch")
124             values = cached[["raw_lag", "arcsinh_lag"]].to_numpy(dtype=np.float64)
125             seconds = float(cached.fold_seconds.iloc[0])
126             print(f"{target} fold {fold + 1}/5: loaded checkpoint", flush=True)
127         else:
128             started = time.perf_counter()
129             fit_rows, valid_rows = train.iloc[fit_idx], train.iloc[valid_idx]
130             x_train, x_valid = matrices(
131                 fit_rows, valid_rows, features, numeric, categorical, scaled=False
132             )
133             y_train = fit_rows[target].to_numpy(dtype=np.float64)
134             raw = fit_lightgbm(x_train, y_train, x_valid, fold=fold, transform=False)
135             transformed = fit_lightgbm(x_train, y_train, x_valid, fold=fold, transform=True)
136             values = np.column_stack([raw, transformed])
137             seconds = time.perf_counter() - started
138             cached = pd.DataFrame({"row_index": valid_idx,
139                                   ID_COLUMN: train.iloc[valid_idx][ID_COLUMN].to_numpy(),
140                                   "raw_lag": raw, "arcsinh_lag": transformed,
141                                   "fold_seconds": seconds})
```

Code appendix | hard_target_oof.py

Fixed lag features and target-transform experiments

```
142         cached.to_parquet(path, index=False, compression="zstd")
143         print(f"{target} fold {fold + 1}/5: {seconds:.1f}s", flush=True)
144         del fit_rows, valid_rows, x_train, x_valid, y_train, cached
145         gc.collect()
146         output[valid_idx] = values
147         fold_times.append(seconds)
148     if not np.isfinite(output).all():
149         raise ValueError("Hard-target OOF matrix incomplete")
150     aligned = pd.DataFrame({"row_index": np.arange(len(train)), ID_COLUMN: train[ID_COLUMN],
151                             "raw_lag": output[:, 0], "arcsinh_lag": output[:, 1]})
152     aligned.to_parquet(OUTPUT / f"hard_{target}_oof.parquet", index=False, compression="zstd")
153     truth = train[list(TARGETS)]
154     actual = train[target].to_numpy(dtype=np.float64)
155     metrics = {"target": target, "config": CONFIG, "engineered_features": list(engineered.columns),
156               "fold_seconds": fold_times}
157     for j, method in enumerate(("raw_lag", "arcsinh_lag")):
158         metrics[method] = {
159             "smape_percent": target_score(truth, target, output[:, j]),
160             "sign_accuracy_percent": 100.0 * float(np.mean(np.sign(actual) == np.sign(output[:, j]))),
161             "prediction_min": float(output[:, j].min()),
162             "prediction_max": float(output[:, j].max()),
163         }
164     (OUTPUT / f"hard_{target}_metrics.json").write_text(json.dumps(metrics, indent=2) + "\n")
165     print(f"{target}: raw+lags={metrics['raw_lag']['smape_percent']:.4f}%, "
166           f"arcsinh+lags={metrics['arcsinh_lag']['smape_percent']:.4f}%", flush=True)
167
168
169 def main() -> None:
170     parser = argparse.ArgumentParser()
171     group = parser.add_mutually_exclusive_group(required=True)
172     group.add_argument("--target", choices=HARD_TARGETS)
173     group.add_argument("--all", action="store_true")
174     parser.add_argument("--force", action="store_true")
175     args = parser.parse_args()
176     if args.all:
177         for target in HARD_TARGETS:
178             command = [sys.executable, str(Path(__file__).resolve()), "--target", target]
179             if args.force:
180                 command.append("--force")
181             subprocess.run(command, check=True)
182     else:
183         run_target(args.target, force=args.force)
184
185
186 if __name__ == "__main__":
187     main()
```

Code appendix | analyze_oof.py

Lag baselines, target-specific selection, accounting identities

```
001  """Analyze aligned OOF models, lag baselines, target blends, and identities.
002
003  Only train.csv and saved OOF predictions are read. Never reads test_individual.csv.
004  Run --prelim for lag/identity checks, then --all after model OOF files exist.
005  """
006
007  from __future__ import annotations
008
009  import argparse
010  import json
011  from pathlib import Path
012
013  import numpy as np
014  import pandas as pd
015
016  from evaluation_framework import ID_COLUMN, ROOT, TARGETS, score_predictions
017  from hard_target_oof import HARD_TARGETS
018
019  OUTPUT = ROOT / "oof_outputs"
020  LAG_METHODS = ("q1", "mean_q1_q2", "mean_q1_q4", "weighted_q1_q4")
021  RF_WEIGHTS = (0.0, 0.25, 0.5, 0.75, 1.0)
022  LAG_BLEND_WEIGHTS = (0.25, 0.5, 0.75)
023  DIRECT_WEIGHTS = (0.0, 0.25, 0.5, 0.75, 1.0)
024
025
026  def read_training_subset() -> pd.DataFrame:
027      columns = [ID_COLUMN, *TARGETS]
028      columns += [f"Q{k}_{target.removeprefix('Q0_')}" for target in TARGETS for k in range(1, 5)]
029      data = pd.read_csv(ROOT / "train.csv", usecols=columns)
030      if len(data) != 100_000 or not data[ID_COLUMN].is_unique:
031          raise ValueError("Unexpected training row count or duplicate Id")
032      return data
033
034
035  def target_score(truth: pd.DataFrame, target: str, prediction: np.ndarray) -> float:
036      """Score one candidate through the exact shared nine-target rubric function."""
037      surrogate = truth.copy(deep=False)
038      surrogate[target] = np.asarray(prediction, dtype=np.float64)
039      per_target, _ = score_predictions(truth, surrogate)
040      return float(per_target[target])
041
042
043  def aligned_frame(data: pd.DataFrame, prediction: pd.DataFrame) -> pd.DataFrame:
044      aligned = prediction.copy()
045      aligned.insert(0, ID_COLUMN, data[ID_COLUMN].to_numpy())
046      aligned.insert(0, "row_index", np.arange(len(data)))
047      return aligned
```

Code appendix | analyze_oof.py

Lag baselines, target-specific selection, accounting identities

```
048
049
050 def read_aligned(path: Path, data: pd.DataFrame, columns: list[str]) -> pd.DataFrame:
051     saved = pd.read_parquet(path)
052     if not np.array_equal(saved.row_index.to_numpy(), np.arange(len(data))) or not np.array_equal(
053         saved[ID_COLUMN].to_numpy(), data[ID_COLUMN].to_numpy()
054     ):
055         raise ValueError(f"00F row/Id mismatch: {path}")
056     if not np.isfinite(saved[columns].to_numpy(dtype=np.float64)).all():
057         raise ValueError(f"Non-finite 00F prediction: {path}")
058     return saved[columns]
059
060
061 def lag_predictions(data: pd.DataFrame) -> dict[str, pd.DataFrame]:
062     outputs = {name: pd.DataFrame(index=data.index, columns=TARGETS, dtype=np.float64)
063               for name in LAG_METHODS}
064     weights = np.array([0.6, 0.25, 0.10, 0.05])
065     for target in TARGETS:
066         suffix = target.removeprefix("Q0_")
067         lag = data[[f"Q{k}_{suffix}" for k in range(1, 5)]].to_numpy(dtype=np.float64)
068         outputs["q1"][target] = lag[:, 0]
069         outputs["mean_q1_q2"][target] = lag[:, :2].mean(axis=1)
070         outputs["mean_q1_q4"][target] = lag.mean(axis=1)
071         outputs["weighted_q1_q4"][target] = lag @ weights
072     return outputs
073
074
075 def identity_checks(data: pd.DataFrame, truth: pd.DataFrame) -> pd.DataFrame:
076     relationships = [
077         ("assets = liabilities + equity", "Q0_TOTAL_ASSETS",
078          data.Q0_TOTAL_LIABILITIES + data.Q0_TOTAL_STOCKHOLDERS_EQUITY),
079         ("revenues = gross_profit + cost_of_revenues", "Q0_REVENUES",
080          data.Q0_GROSS_PROFIT + data.Q0_COST_OF_REVENUES),
081         ("operating_income = gross_profit - operating_expenses", "Q0_OPERATING_INCOME",
082          data.Q0_GROSS_PROFIT - data.Q0_OPERATING_EXPENSES),
083     ]
084     rows = []
085     for identity, target, derived in relationships:
086         residual = data[target] - derived
087         rows.append({
088             "identity": identity,
089             "target": target,
090             "identity_smape_percent": target_score(truth, target, derived.to_numpy()),
091             "median_absolute_residual": float(np.median(np.abs(residual))),
092             "p95_absolute_residual": float(np.quantile(np.abs(residual), 0.95)),
093             "max_absolute_residual": float(np.max(np.abs(residual))),
094         })
```

Code appendix | analyze_oof.py

Lag baselines, target-specific selection, accounting identities

```
095     return pd.DataFrame(rows)
096
097
098 def run_prelim(data: pd.DataFrame) -> dict[str, pd.DataFrame]:
099     OUTPUT.mkdir(exist_ok=True)
100     truth = data[list(TARGETS)]
101     lag = lag_predictions(data)
102     rows = []
103     for name, prediction in lag.items():
104         per_target, average = score_predictions(truth, prediction)
105         row = {"method": name, "mean_smape_percent": average}
106         row.update({target: float(per_target[target]) for target in TARGETS})
107         rows.append(row)
108         aligned_frame(data, prediction).to_parquet(
109             OUTPUT / f"lag_{name}.parquet", index=False, compression="zstd"
110         )
111     pd.DataFrame(rows).sort_values("mean_smape_percent").to_csv(
112         OUTPUT / "lag_scores.csv", index=False
113     )
114     identity_checks(data, truth).to_csv(OUTPUT / "identity_checks.csv", index=False)
115     print("Lag baselines:\n", pd.DataFrame(rows)[["method", "mean_smape_percent"]].to_string(index=False),
116           flush=True)
117     return lag
118
119
120 def load_model_oof(data: pd.DataFrame, truth: pd.DataFrame) -> dict[str, pd.DataFrame]:
121     models = {}
122     rows = []
123     for name in ("random_forest", "lightgbm", "hist_gradient_boosting"):
124         prediction = read_aligned(OUTPUT / f"{name}_oof.parquet", data, list(TARGETS))
125         per_target, mean = score_predictions(truth, prediction)
126         recorded = json.loads((OUTPUT / f"{name}_metrics.json").read_text())
127         if abs(mean - recorded["mean_smape_percent"]) > 1e-9:
128             raise ValueError(f"Saved score mismatch: {name}")
129         models[name] = prediction
130         row = {"model": name, "mean_smape_percent": mean}
131         row.update({target: float(per_target[target]) for target in TARGETS})
132         rows.append(row)
133     pd.DataFrame(rows).sort_values("mean_smape_percent").to_csv(
134         OUTPUT / "base_model_scores.csv", index=False
135     )
136     return models
137
138
139 def load_hard_oof(data: pd.DataFrame, truth: pd.DataFrame,
140                  models: dict[str, pd.DataFrame]) -> dict[str, pd.DataFrame]:
141     hard = {}
```

Code appendix | analyze_oof.py

Lag baselines, target-specific selection, accounting identities

```
142     rows = []
143     fold_rows = []
144     assignment = pd.read_parquet(OUTPUT / "fold_assignments.parquet")
145     if not np.array_equal(assignment.row_index.to_numpy(), np.arange(len(data))) or not np.array_equal(
146         assignment[ID_COLUMN].to_numpy(), data[ID_COLUMN].to_numpy()
147     ):
148         raise ValueError("Fold assignments are misaligned")
149     fold_ids = assignment.fold.to_numpy()
150     for target in HARD_TARGETS:
151         saved = read_aligned(OUTPUT / f"hard_{target}_oof.parquet",
152                               data, ["raw_lag", "arcsinh_lag"])
153         hard[target] = saved
154         actual = truth[target].to_numpy(dtype=np.float64)
155         for method, prediction in (
156             ("lightgbm_base_raw", models["lightgbm"][target].to_numpy()),
157             ("lightgbm_raw_lag", saved.raw_lag.to_numpy()),
158             ("lightgbm_arcsinh_lag", saved.arcsinh_lag.to_numpy()),
159         ):
160             row = {"target": target, "method": method,
161                   "smape_percent": target_score(truth, target, prediction)}
162             if target in ("Q0_OPERATING_INCOME", "Q0_EBITDA"):
163                 row["sign_accuracy_percent"] = 100 * float(np.mean(
164                     np.sign(actual) == np.sign(prediction)
165                 ))
166             rows.append(row)
167             for fold in range(5):
168                 inside = fold_ids == fold
169                 fold_truth = truth.loc[inside].reset_index(drop=True)
170                 fold_rows.append({"target": target, "method": method, "fold": fold + 1,
171                                   "smape_percent": target_score(fold_truth, target,
172                                                                 prediction[inside])})
173     pd.DataFrame(rows).to_csv(OUTPUT / "hard_experiment_scores.csv", index=False)
174     pd.DataFrame(fold_rows).to_csv(OUTPUT / "hard_fold_scores.csv", index=False)
175     return hard
176
177
178 def select_direct(data: pd.DataFrame, truth: pd.DataFrame,
179                  models: dict[str, pd.DataFrame], lag: dict[str, pd.DataFrame],
180                  hard: dict[str, pd.DataFrame]) -> tuple[pd.DataFrame, pd.DataFrame]:
181     selected = pd.DataFrame(index=data.index, columns=TARGETS, dtype=np.float64)
182     selections = []
183     all_candidates = []
184     for target in TARGETS:
185         rf = models["random_forest"][target].to_numpy(dtype=np.float64)
186         lgb = models["lightgbm"][target].to_numpy(dtype=np.float64)
187         candidates: list[tuple[str, np.ndarray, dict]] = []
188         rf_lgb = []
```

Code appendix | analyze_oof.py

Lag baselines, target-specific selection, accounting identities

```
189     for rf_weight in RF_WEIGHTS:
190         prediction = rf_weight * rf + (1 - rf_weight) * lgb
191         label = f"rf_lgb_rf_weight_{rf_weight:.2f}"
192         score = target_score(truth, target, prediction)
193         rf_lgb.append((score, rf_weight, prediction))
194         candidates.append((label, prediction, {"rf_weight": rf_weight,
195                                             "lgb_weight": 1 - rf_weight}))
196     best_blend_score, best_rf_weight, best_blend_prediction = min(rf_lgb, key=lambda x: x[0])
197     candidates.append(("hist_gradient_boosting",
198                       models["hist_gradient_boosting"][target].to_numpy(dtype=np.float64), {}))
199     for lag_name in LAG_METHODS:
200         lag_value = lag[lag_name][target].to_numpy(dtype=np.float64)
201         candidates.append((f"lag_{lag_name}", lag_value,
202                           {"lag_method": lag_name, "lag_weight": 1.0}))
203     for lag_weight in LAG_BLEND_WEIGHTS:
204         prediction = lag_weight * lag_value + (1 - lag_weight) * best_blend_prediction
205         candidates.append((f"lag_{lag_name}_w{lag_weight:.2f}_plus_rf_lgb",
206                           prediction, {"lag_method": lag_name, "lag_weight": lag_weight,
207                                       "rf_weight_in_other_part": best_rf_weight}))
208     if target in hard:
209         candidates.append(("lightgbm_raw_lag", hard[target].raw_lag.to_numpy(dtype=np.float64), {}))
210         candidates.append(("lightgbm_arcsinh_lag",
211                           hard[target].arcsinh_lag.to_numpy(dtype=np.float64), {}))
212     evaluated = []
213     for label, prediction, parameters in candidates:
214         score = target_score(truth, target, prediction)
215         all_candidates.append({"target": target, "method": label,
216                               "smape_percent": score, **parameters})
217     evaluated.append((score, label, prediction, parameters))
218     score, label, prediction, parameters = min(evaluated, key=lambda item: item[0])
219     selected[target] = prediction
220     selections.append({"target": target, "best_rf_weight": best_rf_weight,
221                      "best_rf_lgb_smape_percent": best_blend_score,
222                      "best_direct_method": label,
223                      "best_direct_smape_percent": score,
224                      "best_direct_parameters": json.dumps(parameters, sort_keys=True)})
225     pd.DataFrame(all_candidates).sort_values(["target", "smape_percent"]).to_csv(
226         OUTPUT / "direct_candidates.csv", index=False
227     )
228     selection_table = pd.DataFrame(selections)
229     selection_table.to_csv(OUTPUT / "direct_selection.csv", index=False)
230     aligned_frame(data, selected).to_parquet(
231         OUTPUT / "selected_direct_oof.parquet", index=False, compression="zstd"
232     )
233     return selected, selection_table
234
235
```

Code appendix | analyze_oof.py

Lag baselines, target-specific selection, accounting identities

```
236 def accounting_derivations(direct: pd.DataFrame) -> dict[str, list[tuple[str, np.ndarray]]]:
237     d = direct
238     return {
239         "Q0_TOTAL_ASSETS": [(("liabilities_plus_equity",
240                               (d.Q0_TOTAL_LIABILITIES + d.Q0_TOTAL_STOCKHOLDERS_EQUITY).to_numpy())],
241         "Q0_TOTAL_LIABILITIES": [(("assets_minus_equity",
242                                    (d.Q0_TOTAL_ASSETS - d.Q0_TOTAL_STOCKHOLDERS_EQUITY).to_numpy())],
243         "Q0_TOTAL_STOCKHOLDERS_EQUITY": [(("assets_minus_liabilities",
244                                             (d.Q0_TOTAL_ASSETS - d.Q0_TOTAL_LIABILITIES).to_numpy())],
245         "Q0_REVENUES": [(("gross_profit_plus_cost",
246                           (d.Q0_GROSS_PROFIT + d.Q0_COST_OF_REVENUES).to_numpy())],
247         "Q0_GROSS_PROFIT": [
248             ("revenues_minus_cost", (d.Q0_REVENUES - d.Q0_COST_OF_REVENUES).to_numpy()),
249             ("operating_income_plus_expenses",
250              (d.Q0_OPERATING_INCOME + d.Q0_OPERATING_EXPENSES).to_numpy()),
251         ],
252         "Q0_COST_OF_REVENUES": [(("revenues_minus_gross_profit",
253                                   (d.Q0_REVENUES - d.Q0_GROSS_PROFIT).to_numpy())],
254         "Q0_OPERATING_INCOME": [(("gross_profit_minus_expenses",
255                                    (d.Q0_GROSS_PROFIT - d.Q0_OPERATING_EXPENSES).to_numpy())],
256         "Q0_OPERATING_EXPENSES": [(("gross_profit_minus_operating_income",
257                                      (d.Q0_GROSS_PROFIT - d.Q0_OPERATING_INCOME).to_numpy())],
258     }
259
260
261 def accounting_model_experiment(truth: pd.DataFrame,
262                                models: dict[str, pd.DataFrame]) -> None:
263     """Direct/derived/grid blends using each base model's OOF values alone."""
264     rows = []
265     for model_name, prediction in models.items():
266         for target, formulas in accounting_derivations(prediction).items():
267             direct = prediction[target].to_numpy(dtype=np.float64)
268             for formula, derived in formulas:
269                 for weight in DIRECT_WEIGHTS:
270                     candidate = weight * direct + (1 - weight) * derived
271                     rows.append({"model": model_name, "target": target,
272                                 "derived_formula": formula,
273                                 "direct_weight": weight,
274                                 "smape_percent": target_score(truth, target, candidate)})
275     pd.DataFrame(rows).to_csv(OUTPUT / "accounting_model_candidates.csv", index=False)
276
277
278 def select_accounting(data: pd.DataFrame, truth: pd.DataFrame,
279                       direct: pd.DataFrame, direct_selection: pd.DataFrame) -> pd.DataFrame:
280     derived = accounting_derivations(direct)
281     final = direct.copy()
282     candidates = []
```

Code appendix | analyze_oof.py

Lag baselines, target-specific selection, accounting identities

```
283 recipes = []
284 direct_lookup = direct_selection.set_index("target")
285 for target in TARGETS:
286     direct_value = direct[target].to_numpy(dtype=np.float64)
287     direct_score = target_score(truth, target, direct_value)
288     best = (direct_score, "direct", 1.0, direct_value)
289     candidates.append({"target": target, "derived_formula": "direct",
290                       "direct_weight": 1.0, "smape_percent": direct_score})
291     for formula, derived_value in derived.get(target, []):
292         for weight in DIRECT_WEIGHTS:
293             prediction = weight * direct_value + (1 - weight) * derived_value
294             score = target_score(truth, target, prediction)
295             candidates.append({"target": target, "derived_formula": formula,
296                               "direct_weight": weight, "smape_percent": score})
297             if score < best[0]:
298                 best = (score, formula, weight, prediction)
299 score, formula, weight, prediction = best
300 nonnegative_guard = target in ("Q0_REVENUES", "Q0_OPERATING_EXPENSES")
301 clipped_count = int((prediction < 0).sum()) if nonnegative_guard else 0
302 if nonnegative_guard:
303     # These two labels are nonnegative in every training fold. For a
304     # positive actual, a negative or zero prediction both score 200%.
305     prediction = np.maximum(prediction, 0.0)
306     score = target_score(truth, target, prediction)
307 final[target] = prediction
308 recipes.append({
309     "target": target,
310     "direct_method": direct_lookup.loc[target, "best_direct_method"],
311     "direct_parameters": direct_lookup.loc[target, "best_direct_parameters"],
312     "direct_smape_percent": direct_score,
313     "derived_formula": formula,
314     "direct_weight": weight,
315     "nonnegative_guard": nonnegative_guard,
316     "clipped_negative_count": clipped_count,
317     "final_smape_percent": score,
318 })
319 pd.DataFrame(candidates).sort_values(["target", "smape_percent"]).to_csv(
320     OUTPUT / "accounting_candidates.csv", index=False
321 )
322 recipe = pd.DataFrame(recipes)
323 recipe.to_csv(OUTPUT / "target_recipes.csv", index=False)
324 aligned_frame(data, final).to_parquet(
325     OUTPUT / "selected_final_oof.parquet", index=False, compression="zstd"
326 )
327 direct_per_target, direct_mean = score_predictions(truth, direct)
328 final_per_target, final_mean = score_predictions(truth, final)
329 summary = {"direct_selected_mean_smape_percent": direct_mean,
```

Code appendix | analyze_oof.py

Lag baselines, target-specific selection, accounting identities

```
330         "final_selected_mean_smape_percent": final_mean,
331         "direct_target_smape_percent": direct_per_target.to_dict(),
332         "final_target_smape_percent": final_per_target.to_dict(),
333         "selection_note": "Recipes/weights selected on same OOF labels; post-selection scores optimistic."}
334     (OUTPUT / "selection_summary.json").write_text(json.dumps(summary, indent=2) + "\n")
335     assignment = pd.read_parquet(OUTPUT / "fold_assignments.parquet")
336     fold_rows = []
337     for fold in range(5):
338         inside = assignment.fold.to_numpy() == fold
339         per_target, mean = score_predictions(truth.loc[inside], final.loc[inside])
340         fold_row = {"fold": fold + 1, "mean_smape_percent": mean}
341         fold_row.update({target: float(per_target[target]) for target in TARGETS})
342         fold_rows.append(fold_row)
343     pd.DataFrame(fold_rows).to_csv(OUTPUT / "selected_final_fold_scores.csv", index=False)
344     print(f"Selected direct OOF mean: {direct_mean:.5f}%", flush=True)
345     print(f"Selected after accounting blends: {final_mean:.5f}%", flush=True)
346     return recipe
347
348
349 def main() -> None:
350     parser = argparse.ArgumentParser()
351     group = parser.add_mutually_exclusive_group(required=True)
352     group.add_argument("--prelim", action="store_true")
353     group.add_argument("--all", action="store_true")
354     args = parser.parse_args()
355     data = read_training_subset()
356     lag = run_prelim(data)
357     if args.all:
358         truth = data[list(TARGETS)]
359         models = load_model_oof(data, truth)
360         accounting_model_experiment(truth, models)
361         hard = load_hard_oof(data, truth, models)
362         direct, selection = select_direct(data, truth, models, lag, hard)
363         recipe = select_accounting(data, truth, direct, selection)
364         print(recipe[["target", "direct_method", "derived_formula", "direct_weight",
365                     "final_smape_percent"]].to_string(index=False), flush=True)
366
367
368 if __name__ == "__main__":
369     main()
```

Code appendix | locked_recipe_cv.py

Frozen recipe on new CV seeds and shared reconciliation

```
001  """Re-evaluate the frozen seed-42 recipe on a new seed-2026 five-fold split.
002
003  Only train.csv is read. The recipe hash is fixed below and no choices are
004  re-optimized. Run components separately for memory control, then --assemble.
005  """
006
007  from __future__ import annotations
008
009  import argparse
010  import gc
011  import hashlib
012  import json
013  import os
014  import shutil
015  import time
016  from pathlib import Path
017
018  import numpy as np
019  import pandas as pd
020  from sklearn.model_selection import KFold
021
022  from analyze_oof import accounting_derivations
023  from benchmark_models import matrices
024  from evaluation_framework import ID_COLUMN, ROOT, TARGETS, predictor_columns, score_predictions
025  from hard_target_oof import HARD_TARGETS, fit_lightgbm, lag_features
026  from oof_models import train_predict_fold
027
028  SEED = int(os.environ.get("LOCKED_CV_SEED", "2026"))
029  N_SPLITS = 5
030  OUTPUT = ROOT / os.environ.get("LOCKED_CV_OUTPUT", f"locked_cv_{SEED}")
031  CACHE = OUTPUT / "fold_cache"
032  SOURCE_RECIPE = ROOT / "oof_outputs" / "target_recipes.csv"
033  FROZEN_RECIPE = OUTPUT / "target_recipes_frozen.csv"
034  RECIPE_SHA256 = "c7546ae4a87d6b561156cd0d050085e3485a12865db604ac1e0a2385640d8a34"
035
036
037  def freeze_recipe() -> pd.DataFrame:
038      OUTPUT.mkdir(exist_ok=True)
039      if not FROZEN_RECIPE.exists():
040          if hashlib.sha256(SOURCE_RECIPE.read_bytes()).hexdigest() != RECIPE_SHA256:
041              raise ValueError("Original recipe changed before freezing")
042          shutil.copyfile(SOURCE_RECIPE, FROZEN_RECIPE)
043          if hashlib.sha256(FROZEN_RECIPE.read_bytes()).hexdigest() != RECIPE_SHA256:
044              raise ValueError("Frozen recipe hash mismatch")
045          recipe = pd.read_csv(FROZEN_RECIPE)
046          if set(recipe.target) != set(TARGETS) or len(recipe) != len(TARGETS):
047              raise ValueError("Frozen recipe target set is invalid")
```

Code appendix | locked_recipe_cv.py

Frozen recipe on new CV seeds and shared reconciliation

```
048     return recipe.set_index("target").loc[list(TARGETS)]
049
050
051 def load_train() -> pd.DataFrame:
052     data = pd.read_csv(ROOT / "train.csv", low_memory=False)
053     if len(data) != 100_000 or not data[ID_COLUMN].is_unique:
054         raise ValueError("Unexpected training data")
055     return data
056
057
058 def folds(data: pd.DataFrame) -> list[tuple[np.ndarray, np.ndarray]]:
059     split = list(KFold(n_splits=N_SPLITS, shuffle=True, random_state=SEED).split(data))
060     assignment = np.full(len(data), -1, dtype=np.int8)
061     for fold, (_, valid_idx) in enumerate(split):
062         assignment[valid_idx] = fold
063     if (assignment < 0).any():
064         raise AssertionError("Incomplete folds")
065     frame = pd.DataFrame({"row_index": np.arange(len(data)), ID_COLUMN: data[ID_COLUMN],
066                          "fold": assignment})
067     path = OUTPUT / "fold_assignments.parquet"
068     if path.exists():
069         previous = pd.read_parquet(path)
070         if not previous.equals(frame):
071             raise ValueError("Existing fold assignment mismatch")
072     else:
073         frame.to_parquet(path, index=False, compression="zstd")
074     return split
075
076
077 def checked_cache(path: Path, valid_idx: np.ndarray, ids: np.ndarray,
078                  prediction_columns: list[str]) -> np.ndarray | None:
079     if not path.exists():
080         return None
081     cached = pd.read_parquet(path)
082     if not np.array_equal(cached.row_index.to_numpy(), valid_idx) or not np.array_equal(
083         cached[ID_COLUMN].to_numpy(), ids
084     ):
085         raise ValueError(f"Cache alignment mismatch: {path}")
086     return cached[prediction_columns].to_numpy(dtype=np.float64)
087
088
089 def save_cache(path: Path, valid_idx: np.ndarray, ids: np.ndarray,
090               values: np.ndarray, columns: list[str]) -> None:
091     frame = pd.DataFrame(values, columns=columns)
092     frame.insert(0, ID_COLUMN, ids)
093     frame.insert(0, "row_index", valid_idx)
094     frame.to_parquet(path, index=False, compression="zstd")
```

Code appendix | locked_recipe_cv.py

Frozen recipe on new CV seeds and shared reconciliation

```
095
096
097 def save_complete(path: Path, data: pd.DataFrame, values: np.ndarray,
098                  columns: list[str]) -> None:
099     if not np.isfinite(values).all():
100         raise ValueError(f"Incomplete/non-finite predictions: {path}")
101     frame = pd.DataFrame(values, columns=columns)
102     frame.insert(0, ID_COLUMN, data[ID_COLUMN].to_numpy())
103     frame.insert(0, "row_index", np.arange(len(data)))
104     frame.to_parquet(path, index=False, compression="zstd")
105
106
107 def fit_base(component: str, data: pd.DataFrame,
108             split: list[tuple[np.ndarray, np.ndarray]]) -> None:
109     name = {"rf": "random_forest", "lgb": "lightgbm"}[component]
110     features = predictor_columns(data)
111     numeric = [c for c in features if pd.api.types.is_numeric_dtype(data[c])]
112     categorical = [c for c in features if c not in numeric]
113     output = np.full((len(data), len(TARGETS)), np.nan)
114     times = []
115     for fold, (fit_idx, valid_idx) in enumerate(split):
116         path = CACHE / f"{component}_fold{fold + 1}.parquet"
117         ids = data.iloc[valid_idx][ID_COLUMN].to_numpy()
118         values = checked_cache(path, valid_idx, ids, list(TARGETS))
119         if values is None:
120             started = time.perf_counter()
121             fit_rows, valid_rows = data.iloc[fit_idx], data.iloc[valid_idx]
122             x_train, x_valid = matrices(fit_rows, valid_rows, features, numeric, categorical,
123                                       scaled=False)
124             y_train = fit_rows[list(TARGETS)].to_numpy(dtype=np.float64)
125             values = train_predict_fold(name, x_train, y_train, x_valid, fold)
126             seconds = time.perf_counter() - started
127             save_cache(path, valid_idx, ids, values, list(TARGETS))
128             del fit_rows, valid_rows, x_train, x_valid, y_train
129             gc.collect()
130             print(f"locked {component} fold {fold + 1}/5: {seconds:.1f}s", flush=True)
131         else:
132             seconds = 0.0
133             print(f"locked {component} fold {fold + 1}/5: loaded", flush=True)
134         output[valid_idx] = values
135         times.append(seconds)
136     save_complete(OUTPUT / f"{component}_oof.parquet", data, output, list(TARGETS))
137     score, average = score_predictions(data[list(TARGETS)], pd.DataFrame(output, columns=TARGETS))
138     (OUTPUT / f"{component}_metrics.json").write_text(json.dumps({
139         "seed": SEED, "model": name, "mean_smape_percent": average,
140         "target_smape_percent": score.to_dict(), "new_fold_seconds": times,
141     }, indent=2) + "\n")
```

Code appendix | locked_recipe_cv.py

Frozen recipe on new CV seeds and shared reconciliation

```
142     print(f"locked {component} OOF={average:.5f}%", flush=True)
143
144
145 def fit_hard(data: pd.DataFrame,
146             split: list[tuple[np.ndarray, np.ndarray]]) -> None:
147     for target in HARD_TARGETS:
148         engineered = lag_features(data, target)
149         augmented = pd.concat([data, engineered], axis=1)
150         features = predictor_columns(data) + list(engineered.columns)
151         numeric = [c for c in features if pd.api.types.is_numeric_dtype(augmented[c])]
152         categorical = [c for c in features if c not in numeric]
153         output = np.full(len(data), np.nan)
154         for fold, (fit_idx, valid_idx) in enumerate(split):
155             path = CACHE / f"hard_{target}_fold{fold + 1}.parquet"
156             ids = data.iloc[valid_idx][ID_COLUMN].to_numpy()
157             saved = checked_cache(path, valid_idx, ids, [target])
158             if saved is None:
159                 started = time.perf_counter()
160                 fit_rows, valid_rows = augmented.iloc[fit_idx], augmented.iloc[valid_idx]
161                 x_train, x_valid = matrices(fit_rows, valid_rows, features, numeric,
162                                           categorical, scaled=False)
163                 y_train = fit_rows[target].to_numpy(dtype=np.float64)
164                 values = fit_lightgbm(x_train, y_train, x_valid, fold=fold, transform=True)
165                 save_cache(path, valid_idx, ids, values.reshape(-1, 1), [target])
166                 print(f"locked {target} fold {fold + 1}/5: "
167                       f"{time.perf_counter() - started:.1f}s", flush=True)
168                 del fit_rows, valid_rows, x_train, x_valid, y_train
169                 gc.collect()
170             else:
171                 values = saved[:, 0]
172                 print(f"locked {target} fold {fold + 1}/5: loaded", flush=True)
173                 output[valid_idx] = values
174             save_complete(OUTPUT / f"hard_{target}_oof.parquet", data,
175                          output.reshape(-1, 1), [target])
176             del augmented, engineered
177             gc.collect()
178
179
180 def read_aligned(path: Path, data: pd.DataFrame, columns: list[str]) -> pd.DataFrame:
181     frame = pd.read_parquet(path)
182     if not np.array_equal(frame.row_index.to_numpy(), np.arange(len(data))) or not np.array_equal(
183         frame[ID_COLUMN].to_numpy(), data[ID_COLUMN].to_numpy()
184     ):
185         raise ValueError(f"OOF alignment mismatch: {path}")
186     return frame[columns]
187
188
```

Code appendix | locked_recipe_cv.py

Frozen recipe on new CV seeds and shared reconciliation

```
189 def apply_accounting_recipe(direct: pd.DataFrame, recipe: pd.DataFrame) -> pd.DataFrame:
190     """Apply the frozen accounting weights to aligned direct OOF predictions."""
191     derived = accounting_derivations(direct)
192     final = direct.copy()
193     for target, row in recipe.iterrows():
194         formula = row.derived_formula
195         if formula != "direct":
196             sources = dict(derived[target])
197             weight = float(row.direct_weight)
198             final[target] = weight * direct[target] + (1 - weight) * sources[formula]
199         if str(row.nonnegative_guard).lower() == "true":
200             final[target] = np.maximum(final[target], 0.0)
201     return final
202
203
204 def assemble(data: pd.DataFrame, recipe: pd.DataFrame) -> None:
205     rf = read_aligned(OUTPUT / "rf_oof.parquet", data, list(TARGETS))
206     lgb = read_aligned(OUTPUT / "lgb_oof.parquet", data, list(TARGETS))
207     hard = {target: read_aligned(OUTPUT / f"hard_{target}_oof.parquet", data, [target])[target]
208             for target in HARD_TARGETS}
209     direct = pd.DataFrame(index=data.index, columns=TARGETS, dtype=np.float64)
210     for target, row in recipe.iterrows():
211         method = row.direct_method
212         params = json.loads(row.direct_parameters)
213         if method.startswith("rf_lgb_rf_weight"):
214             weight = float(params["rf_weight"])
215             values = weight * rf[target] + (1 - weight) * lgb[target]
216         elif method == "lag_q1":
217             values = data[f"Q1_{target.removeprefix('Q0_')}"]
218         elif method.startswith("lag_q1_w"):
219             lag_weight = float(params["lag_weight"])
220             rf_weight = float(params["rf_weight_in_other_part"])
221             base = rf_weight * rf[target] + (1 - rf_weight) * lgb[target]
222             lag = data[f"Q1_{target.removeprefix('Q0_')}"]
223             values = lag_weight * lag + (1 - lag_weight) * base
224         elif method == "lightgbm_arcsinh_lag":
225             values = hard[target]
226         else:
227             raise ValueError(f"Unrecognized frozen direct method: {method}")
228         direct[target] = values
229
230     final = apply_accounting_recipe(direct, recipe)
231
232     save_complete(OUTPUT / "locked_direct_oof.parquet", data,
233                  direct.to_numpy(dtype=np.float64), list(TARGETS))
234     save_complete(OUTPUT / "locked_final_oof.parquet", data,
235                  final.to_numpy(dtype=np.float64), list(TARGETS))
```

Code appendix | locked_recipe_cv.py

Frozen recipe on new CV seeds and shared reconciliation

```
236     truth = data[list(TARGETS)]
237     per_target, average = score_predictions(truth, final)
238     pd.DataFrame({"target": TARGETS,
239                  "seed42_selected_smape_percent": recipe.final_smape_percent.to_numpy(),
240                  f"seed{SEED}_locked_smape_percent": per_target.to_numpy()}).to_csv(
241         OUTPUT / "locked_target_scores.csv", index=False
242     )
243     assignments = pd.read_parquet(OUTPUT / "fold_assignments.parquet")
244     fold_rows = []
245     for fold in range(N_SPLITS):
246         mask = assignments.fold.to_numpy() == fold
247         scores, mean = score_predictions(truth.loc[mask], final.loc[mask])
248         fold_rows.append({"fold": fold + 1, "mean_smape_percent": mean,
249                          **scores.to_dict()})
250     pd.DataFrame(fold_rows).to_csv(OUTPUT / "locked_fold_scores.csv", index=False)
251     old_average = float(recipe.final_smape_percent.mean())
252     (OUTPUT / "locked_summary.json").write_text(json.dumps({
253         "recipe_sha256": RECIPE_SHA256, "cv_seed": SEED,
254         "old_seed42_selected_mean_smape_percent": old_average,
255         "new_locked_mean_smape_percent": average,
256         "difference_percentage_points": average - old_average,
257         "target_smape_percent": per_target.to_dict(),
258     }, indent=2) + "\n")
259     print(f"Frozen recipe: seed42={old_average:.5f}%, seed{SEED}={average:.5f}%", flush=True)
260
261
262 def main() -> None:
263     parser = argparse.ArgumentParser()
264     parser.add_argument("--component", choices=("rf", "lgb", "hard"))
265     parser.add_argument("--assemble", action="store_true")
266     args = parser.parse_args()
267     if (args.component is None) == (not args.assemble):
268         parser.error("choose exactly one of --component or --assemble")
269     recipe = freeze_recipe()
270     data = load_train()
271     split = folds(data)
272     CACHE.mkdir(exist_ok=True)
273     if args.assemble:
274         assemble(data, recipe)
275     elif args.component == "hard":
276         fit_hard(data, split)
277     else:
278         fit_base(args.component, data, split)
279
280
281 if __name__ == "__main__":
282     main()
```

Code appendix | arcsinh_ablation_cv.py

All-target raw/arcsinh ablation

```
001  """Raw versus arcsinh LightGBM targets for all nine targets on seed-2026 folds.
002
003  Both variants receive identical base features, preprocessing, folds, and model
004  hyperparameters. No test data and no weight/model selection in this script.
005  """
006
007  from __future__ import annotations
008
009  import gc
010  import json
011  import time
012
013  import numpy as np
014  import pandas as pd
015
016  from benchmark_models import matrices
017  from evaluation_framework import ID_COLUMN, ROOT, TARGETS, predictor_columns, score_predictions
018  from hard_target_oof import fit_lightgbm
019  from locked_recipe_cv import checked_cache, folds, load_train, save_cache, save_complete
020
021  OUTPUT = ROOT / "ablation_cv_2026"
022  CACHE = OUTPUT / "fold_cache"
023
024
025  def run() -> None:
026      CACHE.mkdir(parents=True, exist_ok=True)
027      train = load_train()
028      split = folds(train)
029      features = predictor_columns(train)
030      numeric = [c for c in features if pd.api.types.is_numeric_dtype(train[c])]
031      categorical = [c for c in features if c not in numeric]
032      columns = [f"raw_{t}" for t in TARGETS] + [f"arcsinh_{t}" for t in TARGETS]
033      output = np.full((len(train), len(columns)), np.nan, dtype=np.float64)
034      fold_seconds = []
035      for fold, (fit_idx, valid_idx) in enumerate(split):
036          path = CACHE / f"fold{fold + 1}.parquet"
037          ids = train.iloc[valid_idx][ID_COLUMN].to_numpy()
038          values = checked_cache(path, valid_idx, ids, columns)
039          if values is None:
040              started = time.perf_counter()
041              fit_rows, valid_rows = train.iloc[fit_idx], train.iloc[valid_idx]
042              x_train, x_valid = matrices(fit_rows, valid_rows, features, numeric,
043                                       categorical, scaled=False)
044              y_train = fit_rows[list(TARGETS)].to_numpy(dtype=np.float64)
045              raw = np.empty((len(valid_idx), len(TARGETS)), dtype=np.float64)
046              arcsinh = np.empty_like(raw)
047              for j, target in enumerate(TARGETS):
048                  raw[:, j] = fit_lightgbm(x_train, y_train[:, j], x_valid,
049                                          fold=fold, transform=False)
```

Code appendix | arcsinh_ablation_cv.py

All-target raw/arcsinh ablation

```
050         arcsinh[:, j] = fit_lightgbm(x_train, y_train[:, j], x_valid,
051                                     fold=fold, transform=True)
052         print(f"ablation fold {fold + 1}/5: {target}", flush=True)
053         values = np.column_stack([raw, arcsinh])
054         seconds = time.perf_counter() - started
055         save_cache(path, valid_idx, ids, values, columns)
056         del fit_rows, valid_rows, x_train, x_valid, y_train, raw, arcsinh
057         gc.collect()
058         print(f"ablation fold {fold + 1}/5: {seconds:.1f}s", flush=True)
059     else:
060         seconds = 0.0
061         print(f"ablation fold {fold + 1}/5: loaded", flush=True)
062         output[valid_idx] = values
063         fold_seconds.append(seconds)
064     if not np.isfinite(output).all():
065         raise ValueError("Ablation OOF incomplete")
066     save_complete(OUTPUT / "raw_oof.parquet", train, output[:, :len(TARGETS)], list(TARGETS))
067     save_complete(OUTPUT / "arcsinh_oof.parquet", train, output[:, len(TARGETS):], list(TARGETS))
068     truth = train[list(TARGETS)]
069     raw_frame = pd.DataFrame(output[:, :len(TARGETS)], columns=TARGETS)
070     arc_frame = pd.DataFrame(output[:, len(TARGETS):], columns=TARGETS)
071     raw_scores, raw_mean = score_predictions(truth, raw_frame)
072     arc_scores, arc_mean = score_predictions(truth, arc_frame)
073     pd.DataFrame({
074         "target": TARGETS,
075         "raw_smape_percent": raw_scores.to_numpy(),
076         "arcsinh_smape_percent": arc_scores.to_numpy(),
077         "arcsinh_improvement_pp": raw_scores.to_numpy() - arc_scores.to_numpy(),
078     }).to_csv(OUTPUT / "target_scores.csv", index=False)
079     assignment = pd.read_parquet(ROOT / "locked_cv_2026" / "fold_assignments.parquet")
080     fold_rows = []
081     for fold in range(5):
082         mask = assignment.fold.to_numpy() == fold
083         for method, frame in (("raw", raw_frame), ("arcsinh", arc_frame)):
084             scores, mean = score_predictions(truth.loc[mask], frame.loc[mask])
085             fold_rows.append({"fold": fold + 1, "method": method,
086                             "mean_smape_percent": mean, **scores.to_dict()})
087     pd.DataFrame(fold_rows).to_csv(OUTPUT / "fold_scores.csv", index=False)
088     (OUTPUT / "summary.json").write_text(json.dumps({
089         "seed": 2026, "feature_set": "202 base predictors only",
090         "raw_mean_smape_percent": raw_mean,
091         "arcsinh_mean_smape_percent": arc_mean,
092         "new_fold_seconds": fold_seconds,
093     }, indent=2) + "\n")
094     print(f"raw all-target OOF={raw_mean:.5f}%; arcsinh={arc_mean:.5f}%", flush=True)
095
096
097 if __name__ == "__main__":
098     run()
```

Code appendix | hard_representations_cv.py

Margin, sign/magnitude and EBITDA residual experiments

```
001  """OOF hard-target representation experiments on the locked seed-2026 folds.
002
003  All supervised transforms and preprocessing are fitted within each fold.
004  Revenue and operating-income reconstruction uses aligned locked OOF values.
005  """
006
007  from __future__ import annotations
008
009  import gc
010  import os
011  import time
012
013  import numpy as np
014  import pandas as pd
015  from lightgbm import LGBMClassifier, LGBMRegressor
016
017  from benchmark_models import TREE_THREADS, matrices
018  from evaluation_framework import ID_COLUMN, ROOT, TARGETS, predictor_columns, score_predictions
019  from hard_target_oof import lag_features
020  from locked_recipe_cv import (OUTPUT as LOCKED, checked_cache, folds, load_train,
021                               read_aligned, save_cache, save_complete)
022
023  OUTPUT = ROOT / f"representation_cv_{os.environ.get('REPRESENTATION_CV_SEED', '2026')}"
024  CACHE = OUTPUT / "fold_cache"
025  HARD_TARGETS = ("Q0_OPERATING_INCOME", "Q0_EBITDA")
026  METHODS = ("direct_arcsinh", "margin", "sign_magnitude", "residual")
027
028
029  def scale_of(values: np.ndarray) -> float:
030      scale = float(np.median(np.abs(values)))
031      return scale if scale > 0 else 1.0
032
033
034  def estimator(fold: int, *, classifier: bool = False):
035      cls = LGBMClassifier if classifier else LGBMRegressor
036      return cls(n_estimators=120, learning_rate=0.08, num_leaves=31,
037                min_child_samples=40, colsample_bytree=0.8,
038                n_jobs=TREE_THREADS, random_state=42 + fold,
039                deterministic=True, force_col_wise=True, verbosity=-1)
040
041
042  def transformed_fit_predict(x_fit, values, x_valid, fold):
043      scale = scale_of(values)
044      model = estimator(fold)
045      model.fit(x_fit, np.arcsinh(values / scale))
046      return np.sinh(model.predict(x_valid)) * scale
```

Code appendix | hard_representations_cv.py

Margin, sign/magnitude and EBITDA residual experiments

```
047
048
049 def score_one(truth: pd.DataFrame, target: str, values: np.ndarray) -> float:
050     comparison = truth.copy()
051     comparison[target] = values
052     scores, _ = score_predictions(truth, comparison)
053     return float(scores[target])
054
055
056 def run_target(target: str, data: pd.DataFrame, split, locked: pd.DataFrame) -> None:
057     engineered = lag_features(data, target)
058     augmented = pd.concat([data, engineered], axis=1)
059     features = predictor_columns(data) + list(engineered.columns)
060     numeric = [c for c in features if pd.api.types.is_numeric_dtype(augmented[c])]
061     categorical = [c for c in features if c not in numeric]
062     active = list(METHODS if target == "Q0_EBITDA" else METHODS[:-1])
063     output = np.full((len(data), len(active)), np.nan, dtype=np.float64)
064
065     direct = read_aligned(LOCKED / f"hard_{target}_oof.parquet", data, [target])[target].to_numpy()
066     output[:, 0] = direct
067     for fold, (fit_idx, valid_idx) in enumerate(split):
068         path = CACHE / f"{target}_fold{fold + 1}.parquet"
069         ids = data.iloc[valid_idx][ID_COLUMN].to_numpy()
070         predicted = checked_cache(path, valid_idx, ids, active[1:])
071         if predicted is None:
072             started = time.perf_counter()
073             fit_rows, valid_rows = augmented.iloc[fit_idx], augmented.iloc[valid_idx]
074             x_fit, x_valid = matrices(fit_rows, valid_rows, features, numeric,
075                                     categorical, scaled=False)
076             y = fit_rows[target].to_numpy(dtype=np.float64)
077             train_revenue = fit_rows["Q0_REVENUES"].to_numpy(dtype=np.float64)
078             if (train_revenue <= 0).any():
079                 raise ValueError("Revenue is nonpositive; margin definition needs review")
080             margin = y / train_revenue
081             margin_pred = transformed_fit_predict(x_fit, margin, x_valid, fold)
082             revenue_oof = locked.iloc[valid_idx]["Q0_REVENUES"].to_numpy(dtype=np.float64)
083             margin_reconstructed = margin_pred * revenue_oof
084
085             classifier = estimator(fold, classifier=True)
086             classifier.fit(x_fit, (y > 0).astype(np.int8))
087             positive = classifier.predict_proba(x_valid)[: , 1] >= 0.5
088             magnitude = transformed_fit_predict(x_fit, np.abs(y), x_valid, fold)
089             sign_magnitude = np.where(positive, magnitude, -magnitude)
090             parts = [margin_reconstructed, sign_magnitude]
091
092             if target == "Q0_EBITDA":
```

Code appendix | hard_representations_cv.py

Margin, sign/magnitude and EBITDA residual experiments

```
093         residual = y - fit_rows["Q0_OPERATING_INCOME"].to_numpy(dtype=np.float64)
094         residual_pred = transformed_fit_predict(x_fit, residual, x_valid, fold)
095         oi_oof = locked.iloc[valid_idx]["Q0_OPERATING_INCOME"].to_numpy(dtype=np.float64)
096         parts.append(oi_oof + residual_pred)
097     predicted = np.column_stack(parts)
098     if not np.isfinite(predicted).all():
099         raise ValueError("Non-finite hard-target prediction")
100     save_cache(path, valid_idx, ids, predicted, active[1:])
101     print(f"{target} representations fold {fold + 1}/5: "
102           f"{time.perf_counter() - started:.1f}s", flush=True)
103     del fit_rows, valid_rows, x_fit, x_valid, classifier
104     gc.collect()
105 else:
106     print(f"{target} representations fold {fold + 1}/5: loaded", flush=True)
107     output[valid_idx, 1:] = predicted
108
109 save_complete(OUTPUT / f"{target}_oof.parquet", data, output, active)
110 truth = data[list(TARGETS)]
111 assignments = pd.read_parquet(LOCKED / "fold_assignments.parquet").fold.to_numpy()
112 rows, fold_rows = [], []
113 actual = data[target].to_numpy(dtype=np.float64)
114 for j, method in enumerate(active):
115     rows.append({"target": target, "method": method,
116                 "smape_percent": score_one(truth, target, output[:, j]),
117                 "sign_accuracy_percent": 100 * np.mean(np.sign(actual) == np.sign(output[:, j]))})
118     for fold in range(5):
119         mask = assignments == fold
120         fold_rows.append({"target": target, "method": method, "fold": fold + 1,
121                           "smape_percent": score_one(truth.loc[mask], target,
122                                                         output[mask, j])})
123 pd.DataFrame(rows).to_csv(OUTPUT / f"{target}_scores.csv", index=False)
124 pd.DataFrame(fold_rows).to_csv(OUTPUT / f"{target}_fold_scores.csv", index=False)
125 print(pd.DataFrame(rows).to_string(index=False), flush=True)
126
127
128 def main() -> None:
129     CACHE.mkdir(parents=True, exist_ok=True)
130     data = load_train()
131     split = folds(data)
132     locked = read_aligned(LOCKED / "locked_final_oof.parquet", data, list(TARGETS))
133     for target in HARD_TARGETS:
134         run_target(target, data, split, locked)
135
136
137 if __name__ == "__main__":
138     main()
```

Code appendix | lightgbm_small_search.py

Five controlled LightGBM configurations

```
001  """Controlled arcsinh LightGBM search on development folds (seed 2026).
002
003  The search chooses configurations only on these folds. A separate seed is
004  reserved for evaluating the chosen candidates after selection is frozen.
005  """
006
007  from __future__ import annotations
008
009  import gc
010  import json
011  import time
012
013  import numpy as np
014  import pandas as pd
015  from lightgbm import LGBMRegressor
016
017  from benchmark_models import TREE_THREADS, matrices
018  from evaluation_framework import ID_COLUMN, ROOT, TARGETS, predictor_columns, score_predictions
019  from hard_target_oof import HARD_TARGETS, lag_features
020  from locked_recipe_cv import checked_cache, folds, load_train, save_cache, save_complete
021
022  OUTPUT = ROOT / "search_cv_2026"
023  CACHE = OUTPUT / "fold_cache"
024  ABLATION = ROOT / "ablation_cv_2026"
025
026  # Each row alters a controlled aspect of the existing 120-tree configuration.
027  CONFIGS = {
028      "baseline": {"num_leaves": 31, "min_child_samples": 40,
029                  "learning_rate": 0.08, "n_estimators": 120, "objective": "regression"},
030      "small_leaf": {"num_leaves": 15, "min_child_samples": 80,
031                    "learning_rate": 0.08, "n_estimators": 120, "objective": "regression"},
032      "large_leaf": {"num_leaves": 63, "min_child_samples": 20,
033                    "learning_rate": 0.08, "n_estimators": 120, "objective": "regression"},
034      "slow": {"num_leaves": 31, "min_child_samples": 40,
035              "learning_rate": 0.05, "n_estimators": 180, "objective": "regression"},
036      "l1": {"num_leaves": 31, "min_child_samples": 40,
037            "learning_rate": 0.08, "n_estimators": 120, "objective": "regression_l1"},
038  }
039
040
041  def score_one(data: pd.DataFrame, target: str, values: np.ndarray) -> float:
042      truth = data[list(TARGETS)]
043      predicted = truth.copy()
044      predicted[target] = values
045      return float(score_predictions(truth, predicted)[0][target])
046
047
```

Code appendix | lightgbm_small_search.py

Five controlled LightGBM configurations

```
048 def eligible_targets() -> list[str]:
049     scores = pd.read_csv(ABLATION / "target_scores.csv").set_index("target")
050     fold_scores = pd.read_csv(ABLATION / "fold_scores.csv")
051     selected = []
052     for target in TARGETS:
053         pivot = fold_scores.pivot(index="fold", columns="method", values=target)
054         fold_wins = int((pivot["arcsinh"] < pivot["raw"]).sum())
055         improvement = float(scores.loc[target, "arcsinh_improvement_pp"])
056         if improvement >= 0.15 and fold_wins >= 3:
057             selected.append(target)
058     return selected
059
060
061 def run_target(target: str, data: pd.DataFrame, split) -> dict:
062     engineered = lag_features(data, target) if target in HARD_TARGETS else None
063     full = pd.concat([data, engineered], axis=1) if engineered is not None else data
064     features = predictor_columns(data) + (list(engineered.columns) if engineered is not None else [])
065     numeric = [c for c in features if pd.api.types.is_numeric_dtype(full[c])]
066     categorical = [c for c in features if c not in numeric]
067     names = list(CONFIGS)
068     output = np.full((len(data), len(names)), np.nan, dtype=np.float64)
069     assignment = np.full(len(data), -1, dtype=np.int8)
070     for fold, (fit_idx, valid_idx) in enumerate(split):
071         assignment[valid_idx] = fold
072         path = CACHE / f"{target}_fold{fold + 1}.parquet"
073         ids = data.iloc[valid_idx][ID_COLUMN].to_numpy()
074         values = checked_cache(path, valid_idx, ids, names)
075         if values is None:
076             started = time.perf_counter()
077             fit_rows, valid_rows = full.iloc[fit_idx], full.iloc[valid_idx]
078             x_fit, x_valid = matrices(fit_rows, valid_rows, features, numeric,
079                                     categorical, scaled=False)
080             y = fit_rows[target].to_numpy(dtype=np.float64)
081             scale = float(np.median(np.abs(y))) or 1.0
082             transformed = np.arcsinh(y / scale)
083             values = np.empty((len(valid_idx), len(names)), dtype=np.float64)
084             for j, name in enumerate(names):
085                 model = LGBMRegressor(**CONFIGS[name], colsample_bytree=0.8,
086                                     n_jobs=TREE_THREADS, random_state=42 + fold,
087                                     deterministic=True, force_col_wise=True, verbosity=-1)
088                 model.fit(x_fit, transformed)
089                 values[:, j] = np.sinh(model.predict(x_valid)) * scale
090             del model
091             save_cache(path, valid_idx, ids, values, names)
092             print(f"search {target} fold {fold + 1}/5: "
093                   f"{time.perf_counter() - started:.1f}s", flush=True)
094     del fit_rows, valid_rows, x_fit, x_valid, y, transformed
```

Code appendix | lightgbm_small_search.py

Five controlled LightGBM configurations

```
095         gc.collect()
096     else:
097         print(f"search {target} fold {fold + 1}/5: loaded", flush=True)
098         output[valid_idx] = values
099     save_complete(OUTPUT / f"{target}_oof.parquet", data, output, names)
100     rows = []
101     for j, name in enumerate(names):
102         folds_ = [score_one(data.loc[assignment == fold], target,
103                             output[assignment == fold, j]) for fold in range(5)]
104         rows.append({"target": target, "config": name,
105                     "smape_percent": score_one(data, target, output[:, j]),
106                     **{f"fold_{k + 1}_smape_percent": value for k, value in enumerate(folds_)}})
107     results = pd.DataFrame(rows).sort_values("smape_percent")
108     results.to_csv(OUTPUT / f"{target}_scores.csv", index=False)
109     baseline = results.set_index("config").loc["baseline"]
110     best = results.iloc[0]
111     wins = sum(best[f"fold_{k}_smape_percent"] < baseline[f"fold_{k}_smape_percent"]
112               for k in range(1, 6))
113     stable = bool(best["config"] != "baseline" and wins >= 3 and
114                  baseline["smape_percent"] - best["smape_percent"] >= 0.15)
115     selected = str(best["config"]) if stable else "baseline"
116     return {"target": target, "selected_config": selected,
117            "development_smape_percent": float(results.set_index("config").loc[selected, "smape_percent"]),
118            "baseline_smape_percent": float(baseline["smape_percent"]),
119            "fold_wins_vs_baseline": int(wins) if stable else 0,
120            "feature_set": "base + target lags" if engineered is not None else "base"}
121
122
123 def main() -> None:
124     CACHE.mkdir(parents=True, exist_ok=True)
125     data = load_train()
126     split = folds(data)
127     selected = eligible_targets()
128     print("Arcsinh-beneficial targets:", selected, flush=True)
129     decisions = [run_target(target, data, split) for target in selected]
130     pd.DataFrame(decisions).to_csv(OUTPUT / "development_selection.csv", index=False)
131     (OUTPUT / "search_plan.json").write_text(json.dumps({
132         "development_seed": 2026, "reserved_validation_seed": 314159,
133         "eligibility": "arcsinh beats raw by >=0.15 pp and on >=3/5 folds",
134         "selection": "best config only if >=0.15 pp and >=3/5 fold wins versus baseline",
135         "configs": CONFIGS, "eligible_targets": selected}, indent=2) + "\n")
136
137
138 if __name__ == "__main__":
139     main()
```

Code appendix | candidate_recipe_cv.py

Candidate selection and reserved-seed comparison

```
001  """Select a small model substitution on development OOF, then validate on seed 314159.
002
003  The original accounting weights stay fixed. Selection and validation outputs
004  are stored separately; neither stage reads test_individual.csv.
005  """
006
007  from __future__ import annotations
008
009  import argparse
010  import gc
011  import json
012  import time
013
014  import numpy as np
015  import pandas as pd
016  from lightgbm import LGBMRegressor
017  from sklearn.model_selection import KFold
018
019  from benchmark_models import TREE_THREADS, matrices
020  from evaluation_framework import ID_COLUMN, ROOT, TARGETS, predictor_columns, score_predictions
021  from hard_target_oof import HARD_TARGETS, lag_features
022  from hard_representations_cv import estimator as representation_estimator, transformed_fit_predict
023  from lightgbm_small_search import CONFIGS
024  from locked_recipe_cv import apply_accounting_recipe, checked_cache, load_train, read_aligned, save_cache, save_complete
025
026  DEV_LOCKED = ROOT / "locked_cv_2026"
027  HELD_LOCKED = ROOT / "locked_cv_314159"
028  DEV_REP = ROOT / "representation_cv_2026"
029  DEV_ARC = ROOT / "ablation_cv_2026"
030  DEV_SEARCH = ROOT / "search_cv_2026"
031  OUTPUT = ROOT / "candidate_cv_314159"
032  CACHE = OUTPUT / "fold_cache"
033  FROZEN = OUTPUT / "selected_substitutions.json"
034
035
036  def recipe() -> pd.DataFrame:
037      frame = pd.read_csv(ROOT / "oof_outputs" / "target_recipes.csv")
038      return frame.set_index("target").loc[list(TARGETS)]
039
040
041  def method_predictions(data: pd.DataFrame, target: str, method: str,
042                        *, held: bool) -> np.ndarray:
043      if method == "locked":
044          root = HELD_LOCKED if held else DEV_LOCKED
045          return read_aligned(root / "locked_direct_oof.parquet", data, list(TARGETS))[target].to_numpy()
046      if held:
047          return read_aligned(OUTPUT / f"{target}_{method}_oof.parquet", data,
048                             [target])[target].to_numpy()
049      if method == "arc_base":
```

Code appendix | candidate_recipe_cv.py

Candidate selection and reserved-seed comparison

```
050         return read_aligned(DEV_ARC / "arcsinh_oof.parquet", data, list(TARGETS))[target].to_numpy()
051     if method == "sign_magnitude":
052         return read_aligned(DEV_REP / f"{target}_oof.parquet", data,
053                             ["sign_magnitude"])[target].to_numpy()
054     if method.startswith("search_"):
055         config = method.removeprefix("search_")
056         return read_aligned(DEV_SEARCH / f"{target}_oof.parquet", data,
057                             [config])[config].to_numpy()
058     raise ValueError(method)
059
060
061 def fold_mean_scores(data: pd.DataFrame, final: pd.DataFrame,
062                     assignments: np.ndarray) -> list[float]:
063     truth = data[list(TARGETS)]
064     return [score_predictions(truth.loc[assignments == fold],
065                             final.loc[assignments == fold])[1] for fold in range(5)]
066
067
068 def select() -> None:
069     OUTPUT.mkdir(exist_ok=True)
070     data = load_train()
071     weights = recipe()
072     direct = read_aligned(DEV_LOCKED / "locked_direct_oof.parquet", data, list(TARGETS)).copy()
073     assignments = pd.read_parquet(DEV_LOCKED / "fold_assignments.parquet").fold.to_numpy()
074     search = pd.read_csv(DEV_SEARCH / "development_selection.csv").set_index("target")
075     decisions = []
076     for target in TARGETS:
077         choices = ["arc_base"]
078         if target in search.index and search.loc[target, "selected_config"] != "baseline":
079             choices.append("search_" + str(search.loc[target, "selected_config"]))
080         if target in ("Q0_OPERATING_INCOME", "Q0_EBITDA"):
081             choices.append("sign_magnitude")
082         old_final = apply_accounting_recipe(direct, weights)
083         old_score = score_predictions(data[list(TARGETS)], old_final)[1]
084         old_folds = fold_mean_scores(data, old_final, assignments)
085         chosen = "locked"
086         chosen_score = old_score
087         chosen_folds = old_folds
088         for method in choices:
089             trial = direct.copy()
090             trial[target] = method_predictions(data, target, method, held=False)
091             final = apply_accounting_recipe(trial, weights)
092             score = score_predictions(data[list(TARGETS)], final)[1]
093             folds_ = fold_mean_scores(data, final, assignments)
094             wins = sum(a < b for a, b in zip(folds_, old_folds))
095             if old_score - score >= 0.05 and wins >= 3 and score < chosen_score:
096                 chosen, chosen_score, chosen_folds = method, score, folds_
097         if chosen != "locked":
098             direct[target] = method_predictions(data, target, chosen, held=False)
```

Code appendix | candidate_recipe_cv.py

Candidate selection and reserved-seed comparison

```
099     decisions.append({"target": target, "method": chosen,
100                      "dev_mean_smape_before": old_score,
101                      "dev_mean_smape_after": chosen_score,
102                      "fold_wins": sum(a < b for a, b in zip(chosen_folds, old_folds))})
103     print(f"candidate {target}: {chosen} ({old_score:.5f} -> {chosen_score:.5f})", flush=True)
104     final = apply_accounting_recipe(direct, weights)
105     save_complete(OUTPUT / "development_candidate_oof.parquet", data,
106                  final.to_numpy(dtype=np.float64), list(TARGETS))
107     baseline = read_aligned(DEV_LOCKED / "locked_final_oof.parquet", data, list(TARGETS))
108     old_targets, _ = score_predictions(data[list(TARGETS)], baseline)
109     new_targets, _ = score_predictions(data[list(TARGETS)], final)
110     pd.DataFrame({"target": TARGETS,
111                  "locked_smape_percent": old_targets.to_numpy(),
112                  "candidate_smape_percent": new_targets.to_numpy(),
113                  "improvement_pp": old_targets.to_numpy() - new_targets.to_numpy(),
114                  }).to_csv(OUTPUT / "development_target_scores.csv", index=False)
115     old_folds = fold_mean_scores(data, baseline, assignments)
116     new_folds = fold_mean_scores(data, final, assignments)
117     pd.DataFrame({"fold": range(1, 6), "locked_mean_smape_percent": old_folds,
118                  "candidate_mean_smape_percent": new_folds,
119                  "improvement_pp": np.array(old_folds) - np.array(new_folds),
120                  }).to_csv(OUTPUT / "development_fold_scores.csv", index=False)
121     pd.DataFrame(decisions).to_csv(OUTPUT / "development_decisions.csv", index=False)
122     selected = {row["target"]: row["method"] for row in decisions if row["method"] != "locked"}
123     FROZEN.write_text(json.dumps({"development_seed": 2026, "held_seed": 314159,
124                                  "accounting_recipe": "original frozen seed42 weights",
125                                  "selection_rule": "full mean gain >=0.05 pp and >=3/5 fold wins, one pass in rubric target order",
126                                  "substitutions": selected}, indent=2) + "\n")
127     print(f"development full recipe: {score_predictions(data[list(TARGETS)], final)[1]:.5f}%", flush=True)
128
129
130 def arcsinh_model(fold: int, method: str) -> LGBMRegressor:
131     config = (CONFIGS[method.removeprefix("search_")] if method.startswith("search_")
132              else CONFIGS["baseline"])
133     return LGBMRegressor(**config, colsample_bytree=0.8, n_jobs=TREE_THREADS,
134                          random_state=42 + fold, deterministic=True,
135                          force_col_wise=True, verbosity=-1)
136
137
138 def fit_held() -> None:
139     CACHE.mkdir(parents=True, exist_ok=True)
140     data = load_train()
141     choices = json.loads(FROZEN.read_text())["substitutions"]
142     split = list(KFold(n_splits=5, shuffle=True, random_state=314159).split(data))
143     held_assignments = pd.read_parquet(HELD_LOCKED / "fold_assignments.parquet")
144     assignment = np.full(len(data), -1, dtype=np.int8)
145     for fold, (_, valid_idx) in enumerate(split):
146         assignment[valid_idx] = fold
147     if not np.array_equal(assignment, held_assignments.fold.to_numpy()):
```

Code appendix | candidate_recipe_cv.py

Candidate selection and reserved-seed comparison

```
148         raise ValueError("Held split does not match baseline")
149     for target, method in choices.items():
150         engineered = lag_features(data, target) if (method == "sign_magnitude" or
151             (method.startswith("search_") and target in HARD_TARGETS)) else None
152         full = pd.concat([data, engineered], axis=1) if engineered is not None else data
153         features = predictor_columns(data) + (list(engineered.columns) if engineered is not None else [])
154         numeric = [c for c in features if pd.api.types.is_numeric_dtype(full[c])]
155         categorical = [c for c in features if c not in numeric]
156         output = np.full(len(data), np.nan, dtype=np.float64)
157         for fold, (fit_idx, valid_idx) in enumerate(split):
158             path = CACHE / f"{target}_{method}_fold{fold + 1}.parquet"
159             ids = data.iloc[valid_idx][ID_COLUMN].to_numpy()
160             saved = checked_cache(path, valid_idx, ids, [target])
161             if saved is None:
162                 started = time.perf_counter()
163                 fit_rows, valid_rows = full.iloc[fit_idx], full.iloc[valid_idx]
164                 x_fit, x_valid = matrices(fit_rows, valid_rows, features, numeric,
165                     categorical, scaled=False)
166                 y = fit_rows[target].to_numpy(dtype=np.float64)
167                 scale = float(np.median(np.abs(y))) or 1.0
168                 if method == "sign_magnitude":
169                     classifier = representation_estimator(fold, classifier=True)
170                     classifier.fit(x_fit, (y > 0).astype(np.int8))
171                     positive = classifier.predict_proba(x_valid)[:, 1] >= 0.5
172                     magnitude = transformed_fit_predict(x_fit, np.abs(y), x_valid, fold)
173                     values = np.where(positive, magnitude, -magnitude)
174                 else:
175                     model = arcsinh_model(fold, method)
176                     model.fit(x_fit, np.arcsinh(y / scale))
177                     values = np.sinh(model.predict(x_valid)) * scale
178                 save_cache(path, valid_idx, ids, values.reshape(-1, 1), [target])
179                 print(f"held {target} {method} fold {fold + 1}/5: "
180                     f"{time.perf_counter() - started:.1f}s", flush=True)
181                 if method != "sign_magnitude":
182                     del model
183                 del fit_rows, valid_rows, x_fit, x_valid
184                 gc.collect()
185             else:
186                 values = saved[:, 0]
187                 print(f"held {target} {method} fold {fold + 1}/5: loaded", flush=True)
188             output[valid_idx] = values
189         save_complete(OUTPUT / f"{target}_{method}_oof.parquet", data,
190             output.reshape(-1, 1), [target])
191
192
193 def validate() -> None:
194     data = load_train()
195     choices = json.loads(FROZEN.read_text())["substitutions"]
196     direct = read_aligned(HELD_LOCKED / "locked_direct_oof.parquet", data, list(TARGETS)).copy()
```

Code appendix | candidate_recipe_cv.py

Candidate selection and reserved-seed comparison

```
197 baseline = read_aligned(HELD_LOCKED / "locked_final_oof.parquet", data, list(TARGETS))
198 for target, method in choices.items():
199     direct[target] = method_predictions(data, target, method, held=True)
200 candidate = apply_accounting_recipe(direct, recipe())
201 save_complete(OUTPUT / "held_candidate_oof.parquet", data,
202               candidate.to_numpy(dtype=np.float64), list(TARGETS))
203 truth = data[list(TARGETS)]
204 baseline_scores, baseline_mean = score_predictions(truth, baseline)
205 candidate_scores, candidate_mean = score_predictions(truth, candidate)
206 pd.DataFrame({"target": TARGETS,
207               "locked_smape_percent": baseline_scores.to_numpy(),
208               "candidate_smape_percent": candidate_scores.to_numpy(),
209               "improvement_pp": baseline_scores.to_numpy() - candidate_scores.to_numpy(),
210               }).to_csv(OUTPUT / "held_target_scores.csv", index=False)
211 assignments = pd.read_parquet(HELD_LOCKED / "fold_assignments.parquet").fold.to_numpy()
212 rows = []
213 for fold in range(5):
214     mask = assignments == fold
215     old = score_predictions(truth.loc[mask], baseline.loc[mask])[1]
216     new = score_predictions(truth.loc[mask], candidate.loc[mask])[1]
217     rows.append({"fold": fold + 1, "locked_mean_smape_percent": old,
218                 "candidate_mean_smape_percent": new, "improvement_pp": old - new})
219 pd.DataFrame(rows).to_csv(OUTPUT / "held_fold_scores.csv", index=False)
220 (OUTPUT / "held_summary.json").write_text(json.dumps({
221     "seed": 314159, "locked_mean_smape_percent": baseline_mean,
222     "candidate_mean_smape_percent": candidate_mean,
223     "improvement_pp": baseline_mean - candidate_mean,
224     "substitutions": choices}, indent=2) + "\n")
225 print(f"held baseline={baseline_mean:.5f}%, candidate={candidate_mean:.5f}%", flush=True)
226
227
228 def main() -> None:
229     parser = argparse.ArgumentParser()
230     parser.add_argument("stage", choices=("select", "fit-held", "validate"))
231     stage = parser.parse_args().stage
232     if stage == "select":
233         select()
234     elif stage == "fit-held":
235         fit_held()
236     else:
237         validate()
238
239
240 if __name__ == "__main__":
241     main()
```

Code appendix | submission_code.py

Self-contained final full-data prediction pipeline

```
001  """Final individual prediction pipeline; run with `python submission_code.py`.
002
003  Requires train.csv and test_individual.csv beside this script. The recipe is
004  fixed. All fitting uses the 100,000 training rows; test labels are never used.
005  """
006
007  from __future__ import annotations
008
009  import gc
010  from pathlib import Path
011
012  import numpy as np
013  import pandas as pd
014  from lightgbm import LGBMClassifier, LGBMRegressor
015  from sklearn.compose import ColumnTransformer
016  from sklearn.impute import SimpleImputer
017  from sklearn.pipeline import Pipeline
018  from sklearn.preprocessing import OneHotEncoder
019
020
021  # Configuration
022  ROOT = Path(__file__).resolve().parent
023  TRAIN_PATH = ROOT / "train.csv"
024  TEST_PATH = ROOT / "test_individual.csv"
025  OUTPUT_PATH = ROOT / "submission_code_output.csv"
026  ID = "Id"
027  TARGETS = (
028      "Q0_TOTAL_ASSETS",
029      "Q0_TOTAL_LIABILITIES",
030      "Q0_TOTAL_STOCKHOLDERS_EQUITY",
031      "Q0_GROSS_PROFIT",
032      "Q0_COST_OF_REVENUES",
033      "Q0_REVENUES",
034      "Q0_OPERATING_INCOME",
035      "Q0_OPERATING_EXPENSES",
036      "Q0_EBITDA",
037  )
038  LAG_TARGETS = {"Q0_GROSS_PROFIT", "Q0_OPERATING_INCOME", "Q0_EBITDA"}
039  DIRECT_METHODS = {
040      "Q0_TOTAL_ASSETS": "large_leaf",
041      "Q0_TOTAL_LIABILITIES": "large_leaf",
042      "Q0_TOTAL_STOCKHOLDERS_EQUITY": "large_leaf",
043      "Q0_GROSS_PROFIT": "l1",
044      "Q0_COST_OF_REVENUES": "l1",
045      "Q0_OPERATING_INCOME": "sign_magnitude",
046      "Q0_OPERATING_EXPENSES": "large_leaf",
047      "Q0_EBITDA": "sign_magnitude",
```

Code appendix | submission_code.py

Self-contained final full-data prediction pipeline

```
048 }
049 THREADS = 4
050 SEED = 42
051
052
053 # Data loading
054 def load_data() -> tuple[pd.DataFrame, pd.DataFrame, list[str]]:
055     train = pd.read_csv(TRAIN_PATH, low_memory=False)
056     test = pd.read_csv(TEST_PATH, low_memory=False, dtype={ID: "string"})
057     if len(train) != 100_000 or len(test) != 50_000:
058         raise ValueError("Unexpected train or test row count")
059     if any(target not in train.columns for target in TARGETS):
060         raise ValueError("Training targets are missing")
061     if ID not in train.columns or ID not in test.columns:
062         raise ValueError("Id is missing")
063     predictors = [name for name in train.columns if name not in set(TARGETS) | {ID}]
064     if ID in predictors or set(predictors) & set(TARGETS):
065         raise AssertionError("Id or a Q0 target entered the predictors")
066     if predictors != [name for name in test.columns if name != ID]:
067         raise ValueError("Train/test predictor columns or their order differ")
068     if test[ID].isna().any() or not test[ID].is_unique:
069         raise ValueError("Test Id contains missing or duplicate values")
070     if not np.isfinite(train[list(TARGETS)].to_numpy(dtype=np.float64)).all():
071         raise ValueError("Training targets contain non-finite values")
072     return train, test, predictors
073
074
075 # Feature preprocessing
076 def make_matrices(train_rows: pd.DataFrame, test_rows: pd.DataFrame,
077                  predictors: list[str]) -> tuple[np.ndarray, np.ndarray]:
078     """Fit median imputation and one-hot categories on training rows only."""
079     if ID in predictors or set(predictors) & set(TARGETS):
080         raise AssertionError("Id or target entered the feature matrix")
081     numeric = [c for c in predictors if pd.api.types.is_numeric_dtype(train_rows[c])]
082     categorical = [c for c in predictors if c not in numeric]
083     processor = ColumnTransformer(
084         [
085             ("numeric", Pipeline([("impute", SimpleImputer(strategy="median"))])), numeric),
086             ("categorical", Pipeline([
087                 ("impute", SimpleImputer(strategy="most_frequent")),
088                 ("one_hot", OneHotEncoder(handle_unknown="ignore",
089                                         sparse_output=False, dtype=np.float32)),
089             ]), categorical),
090         ],
091         sparse_threshold=0.0,
092     )
093
094     x_train = processor.fit_transform(train_rows[predictors]).astype(np.float32, copy=False)
```

Code appendix | submission_code.py

Self-contained final full-data prediction pipeline

```
095     x_test = processor.transform(test_rows[predictors]).astype(np.float32, copy=False)
096     if not np.isfinite(x_train).all() or not np.isfinite(x_test).all():
097         raise ValueError("Preprocessed features contain non-finite values")
098     return x_train, x_test
099
100
101 # Lag feature engineering
102 def lag_features(rows: pd.DataFrame, target: str) -> pd.DataFrame:
103     """The exact eleven Q1-Q10 features used by the validated pipeline."""
104     suffix = target.removeprefix("Q0_")
105     values = np.column_stack([rows[f"Q{k}_{suffix}"].to_numpy(dtype=np.float64)
106                               for k in range(1, 11)])
107     if not np.isfinite(values).all():
108         raise ValueError(f"Non-finite lag values for {target}")
109     recent = values[:, :4]
110     weights4 = np.arange(3, -1, -1, dtype=np.float64) - 1.5
111     weights10 = np.arange(9, -1, -1, dtype=np.float64) - 4.5
112     stem = target.lower()
113     return pd.DataFrame({
114         f"{stem}_lag_q1": values[:, 0],
115         f"{stem}_q1_minus_q2": values[:, 0] - values[:, 1],
116         f"{stem}_q1_sign": np.sign(values[:, 0]),
117         f"{stem}_q2_sign": np.sign(values[:, 1]),
118         f"{stem}_q1_q2_same_sign": np.sign(values[:, 0]) * np.sign(values[:, 1]),
119         f"{stem}_q1_q4_mean": recent.mean(axis=1),
120         f"{stem}_q1_q4_median": np.median(recent, axis=1),
121         f"{stem}_q1_q4_std": recent.std(axis=1),
122         f"{stem}_q1_q4_slope_recent": recent @ weights4 / np.dot(weights4, weights4),
123         f"{stem}_q1_q10_mean": values.mean(axis=1),
124         f"{stem}_q1_q10_slope_recent": values @ weights10 / np.dot(weights10, weights10),
125     }, index=rows.index)
126
127
128 def matrices_for_target(train: pd.DataFrame, test: pd.DataFrame,
129                        base_predictors: list[str], target: str):
130     if target not in LAG_TARGETS:
131         return make_matrices(train, test, base_predictors)
132     train_lags = lag_features(train, target)
133     test_lags = lag_features(test, target)
134     if list(train_lags.columns) != list(test_lags.columns):
135         raise ValueError(f"Lag feature names differ for {target}")
136     train_rows = pd.concat([train, train_lags], axis=1)
137     test_rows = pd.concat([test, test_lags], axis=1)
138     return make_matrices(train_rows, test_rows,
139                          base_predictors + list(train_lags.columns))
140
141
```

Code appendix | submission_code.py

Self-contained final full-data prediction pipeline

```
142 # Target transformation
143 def target_scale(values: np.ndarray) -> float:
144     scale = float(np.median(np.abs(values)))
145     return scale if scale > 0 else 1.0
146
147
148 def arcsinh_prediction(model: LGBMRegressor, x_train: np.ndarray,
149                        y_train: np.ndarray, x_test: np.ndarray) -> np.ndarray:
150     scale = target_scale(y_train) # Only the full training target determines scale.
151     model.fit(x_train, np.arcsinh(y_train / scale))
152     return np.sinh(model.predict(x_test)) * scale
153
154
155 # LightGBM helpers
156 def regression_model(method: str) -> LGBMRegressor:
157     if method == "large_leaf":
158         config = {"num_leaves": 63, "min_child_samples": 20,
159                  "objective": "regression"}
160     elif method == "l1":
161         config = {"num_leaves": 31, "min_child_samples": 40,
162                  "objective": "regression_l1"}
163     else:
164         raise ValueError(f"Unknown direct regression method: {method}")
165     return LGBMRegressor(
166         **config, learning_rate=0.08, n_estimators=120,
167         colsample_bytree=0.8, n_jobs=THREADS, random_state=SEED,
168         deterministic=True, force_col_wise=True, verbosity=-1,
169     )
170
171
172 def sign_magnitude_model(*, classifier: bool):
173     cls = LGBMClassifier if classifier else LGBMRegressor
174     return cls(
175         n_estimators=120, learning_rate=0.08, num_leaves=31,
176         min_child_samples=40, colsample_bytree=0.8,
177         n_jobs=THREADS, random_state=SEED,
178         deterministic=True, force_col_wise=True, verbosity=-1,
179     )
180
181
182 # Sign + magnitude models
183 def predict_signed_magnitude(x_train: np.ndarray, y_train: np.ndarray,
184                             x_test: np.ndarray) -> np.ndarray:
185     classifier = sign_magnitude_model(classifier=True)
186     classifier.fit(x_train, (y_train > 0).astype(np.int8))
187     positive = classifier.predict_proba(x_test)[ :, 1] >= 0.5
188     magnitude = arcsinh_prediction(
```

Code appendix | submission_code.py

Self-contained final full-data prediction pipeline

```
189         sign_magnitude_model(classifier=False), x_train, np.abs(y_train), x_test
190     )
191     return np.where(positive, magnitude, -magnitude)
192
193
194 # Train final direct models
195 def train_direct_models(train: pd.DataFrame, test: pd.DataFrame,
196                        predictors: list[str]) -> pd.DataFrame:
197     direct = pd.DataFrame(index=test.index, columns=TARGETS, dtype=np.float64)
198     base_x_train, base_x_test = matrices_for_target(
199         train, test, predictors, "Q0_TOTAL_ASSETS"
200     )
201     for target, method in DIRECT_METHODS.items():
202         if target in LAG_TARGETS:
203             x_train, x_test = matrices_for_target(train, test, predictors, target)
204         else:
205             x_train, x_test = base_x_train, base_x_test
206         y_train = train[target].to_numpy(dtype=np.float64)
207         if method == "sign_magnitude":
208             prediction = predict_signed_magnitude(x_train, y_train, x_test)
209         else:
210             prediction = arcsinh_prediction(
211                 regression_model(method), x_train, y_train, x_test
212             )
213         if not np.isfinite(prediction).all():
214             raise ValueError(f"Non-finite direct prediction for {target}")
215         direct[target] = prediction
216         print(f"trained {target}", flush=True)
217         if target in LAG_TARGETS:
218             del x_train, x_test
219             gc.collect()
220     return direct
221
222
223 # Accounting reconciliation
224 def reconcile_direct_predictions(direct: pd.DataFrame) -> pd.DataFrame:
225     """Every formula below reads only direct predictions, never final values."""
226     d_a = direct["Q0_TOTAL_ASSETS"]
227     d_l = direct["Q0_TOTAL_LIABILITIES"]
228     d_e = direct["Q0_TOTAL_STOCKHOLDERS_EQUITY"]
229     d_gp = direct["Q0_GROSS_PROFIT"]
230     d_c = direct["Q0_COST_OF_REVENUES"]
231     d_oi = direct["Q0_OPERATING_INCOME"]
232     d_oe = direct["Q0_OPERATING_EXPENSES"]
233     d_ebitda = direct["Q0_EBITDA"]
234     final = pd.DataFrame(index=direct.index)
235     final["Q0_TOTAL_ASSETS"] = d_a
```

Code appendix | submission_code.py

Self-contained final full-data prediction pipeline

```
236     final["Q0_TOTAL_LIABILITIES"] = 0.50 * d_l + 0.50 * (d_a - d_e)
237     final["Q0_TOTAL_STOCKHOLDERS_EQUITY"] = d_a - d_l
238     final["Q0_GROSS_PROFIT"] = d_gp
239     final["Q0_COST_OF_REVENUES"] = d_c
240     final["Q0_REVENUES"] = np.maximum(0.0, d_gp + d_c)
241     final["Q0_OPERATING_INCOME"] = 0.50 * d_oi + 0.50 * (d_gp - d_oe)
242     final["Q0_OPERATING_EXPENSES"] = np.maximum(
243         0.0, 0.25 * d_oe + 0.75 * (d_gp - d_oi)
244     )
245     final["Q0_EBITDA"] = d_ebitda
246     return final.loc[:, list(TARGETS)]
247
248
249 # Submission validation
250 def validate_submission(submission: pd.DataFrame, original_ids: pd.Series) -> None:
251     if submission.shape != (50_000, 10) or list(submission.columns) != [ID, *TARGETS]:
252         raise ValueError("Submission shape or column order differs from rubric")
253     if not submission[ID].equals(original_ids):
254         raise ValueError("Test Id values or order changed")
255     if submission[ID].isna().any() or not submission[ID].is_unique:
256         raise ValueError("Submission Id has missing or duplicate values")
257     if not np.isfinite(submission[list(TARGETS)].to_numpy(dtype=np.float64)).all():
258         raise ValueError("Submission contains NaN or infinite predictions")
259     if (submission["Q0_REVENUES"] < 0).any():
260         raise ValueError("Revenue predictions are negative")
261     if (submission["Q0_OPERATING_EXPENSES"] < 0).any():
262         raise ValueError("Operating expense predictions are negative")
263
264
265 # Save predictions
266 def main() -> None:
267     train, test, predictors = load_data()
268     direct = train_direct_models(train, test, predictors)
269     final = reconcile_direct_predictions(direct)
270     submission = pd.concat([test[[ID]], final], axis=1)
271     validate_submission(submission, test[ID])
272     submission.to_csv(OUTPUT_PATH, index=False)
273     saved = pd.read_csv(OUTPUT_PATH, low_memory=False, dtype={ID: "string"})
274     validate_submission(saved, test[ID])
275     print(f"saved {OUTPUT_PATH} ({len(saved):,} rows)", flush=True)
276
277
278 if __name__ == "__main__":
279     main()
```

Code appendix | individual_report_outputs/report_uncertainty.py

Paired bootstrap of stored holdout errors

```
001  """Descriptive uncertainty from saved predictions only; no fitting or selection."""
002
003  from pathlib import Path
004  import json
005
006  import numpy as np
007  import pandas as pd
008
009  ROOT = Path(__file__).resolve().parents[1]
010  TARGETS = (
011      "Q0_TOTAL_ASSETS", "Q0_TOTAL_LIABILITIES", "Q0_TOTAL_STOCKHOLDERS_EQUITY",
012      "Q0_GROSS_PROFIT", "Q0_COST_OF_REVENUES", "Q0_REVENUES",
013      "Q0_OPERATING_INCOME", "Q0_OPERATING_EXPENSES", "Q0_EBITDA",
014  )
015  MODELS = ("mean", "lightgbm", "neural_network")
016  BOOTSTRAPS = 2000
017  SEED = 20261006
018
019
020  def main():
021      truth = pd.read_csv(ROOT / "benchmark_outputs/validation_truth.csv")
022      actual = truth[list(TARGETS)].to_numpy(dtype=np.float64)
023      errors = []
024      for model in MODELS:
025          saved = pd.read_csv(ROOT / f"benchmark_outputs/validation_predictions/{model}.csv")
026          if not saved[["row_index", "Id"]].equals(truth[["row_index", "Id"]]):
027              raise ValueError("Saved validation prediction alignment differs")
028          prediction = saved[list(TARGETS)].to_numpy(dtype=np.float64)
029          denominator = 0.5 * (np.abs(actual) + np.abs(prediction))
030          terms = np.divide(np.abs(actual - prediction), denominator,
031                          out=np.zeros_like(denominator), where=denominator != 0)
032          errors.append(100.0 * terms.mean(axis=1))
033      row_errors = np.column_stack(errors)
034      rng = np.random.default_rng(SEED)
035      boot = np.empty((BOOTSTRAPS, len(MODELS)))
036      for start in range(0, BOOTSTRAPS, 50):
037          stop = min(start + 50, BOOTSTRAPS)
```

Code appendix | individual_report_outputs/report_uncertainty.py

Paired bootstrap of stored holdout errors

```
038         indices = rng.integers(0, len(truth), size=(stop - start, len(truth)))
039         boot[start:stop] = row_errors[indices].mean(axis=1)
040     result = {"seed": SEED, "bootstrap_resamples": BOOTSTRAPS,
041             "validation_rows": len(truth), "models": {}}
042     for j, model in enumerate(MODELS):
043         ci = np.quantile(boot[:, j], [0.025, 0.975])
044         result["models"][model] = {
045             "mean_smape_percent": float(row_errors[:, j].mean()),
046             "conditional_bootstrap_95_interval_percent": ci.tolist(),
047         }
048     difference = boot[:, 2] - boot[:, 1]
049     result["neural_minus_lightgbm"] = {
050         "difference_percentage_points": float((row_errors[:, 2] - row_errors[:, 1]).mean()),
051         "paired_bootstrap_95_interval_percentage_points": np.quantile(
052             difference, [0.025, 0.975]).tolist(),
053     }
054     folds = pd.read_csv(ROOT / "candidate_cv_314159/held_fold_scores.csv")
055     for name in ("locked", "candidate"):
056         values = folds[f"{name}_mean_smape_percent"].to_numpy()
057         result[f"{name}_seed314159_fold_summary"] = {
058             "mean_smape_percent": float(values.mean()),
059             "sample_sd_percentage_points": float(values.std(ddof=1)),
060             "range_percent": [float(values.min()), float(values.max())],
061         }
062     result["limitations"] = (
063         "Bootstrap intervals condition on fixed fitted models and exchangeable validation rows; "
064         "they do not include model selection/training variability. Different CV seeds reuse "
065         "the same rows and are not independent holdout samples."
066     )
067     (ROOT / "individual_report_outputs/uncertainty_summary.json").write_text(
068         json.dumps(result, indent=2) + "\n"
069     )
070     print(json.dumps(result, indent=2))
071
072
073 if __name__ == "__main__":
074     main()
```